# Introducción

Python es un **lenguaje de programación** interpretado multiparadigma, es decir, soporta la programación orientada a objetos, la programación imperativa y, en menor medida, la programación funcional.

En esta unidad vamos a realizar una introducción a **Python** y los requisitos necesarios para su utilización, además vamos a profundizar en las librerías para el análisis de datos en Python.

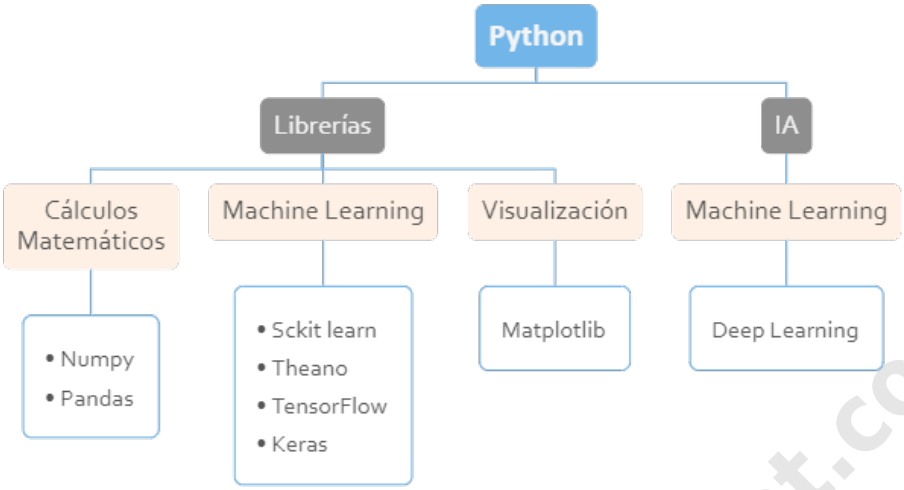
formaciononline.infosernt.com

# Objetivos

- Entender el uso de diferentes lenguajes de programación para el análisis de datos.
- Aprender los conceptos principales de Python.
- Conocer en profundidad las librerías para el análisis de datos en Python.
- Estudiar los fundamentos del Machine Learning.

formaciononline.infosernt.com

# Mapa Conceptual




formaciononline.infosernt.com

# Introducción al Python

El **análisis de datos** se ha convertido actualmente en un proceso importante dentro de las empresas, las cuales requieren identificar variables vitales de sus consumidores para poder ofrecer mejores servicios a estos. Como hemos visto, conocer las tendencias del mercado y conocer porqué suceden ciertas situaciones en las empresas son la base para tomar las decisiones correctas.

Python, por ser un lenguaje muy flexible, ofrece herramientas para realizar el procesamiento de datos de manera muy precisa. Es un lenguaje de programación interpretado multiparadigma, es decir, soporta la programación orientada a objetos, la programación imperativa y, en menor medida, la programación funcional. Fue creado por Guido Van Rossum, usa tipado dinámico, es multiplataforma y debido a que es de código libre, la comunidad de desarrolladores ha creado librerías o módulos para hacer casi cualquier cosa. Su filosofía hace hincapié en una sintaxis que favorezca un código legible, de ahí que la curva de aprendizaje de este lenguaje de programación sea sencilla y rápida.

Por tanto, Python es un lenguaje de programación poderoso y fácil de aprender. Cuenta con estructuras de datos eficientes y de alto nivel y un enfoque simple pero efectivo a la programación orientada a objetos. La elegante sintaxis de Python y su tipado dinámico, junto con su naturaleza interpretada, hacen de éste un lenguaje ideal para scripting y desarrollo rápido de aplicaciones en diversas áreas y sobre la mayoría de plataformas.

## 1.1. Instalación de Python

Como ya hemos visto en el punto anterior, Python es multiplataforma, por lo que funciona tanto en Windows, como MacOs o Linux.

Los **archivos de python** tienen la extensión .py, son archivos de texto que son interpretados por el compilador. Para poder ejecutar programas en Python, necesitamos en primer lugar el intérprete de Python, y en segundo lugar el código que queremos ejecutar.

Se debe tener en cuenta que Python es “case sensitive” lo que quiere decir que diferencia entre mayúsculas y minúsculas.

**Instalación**

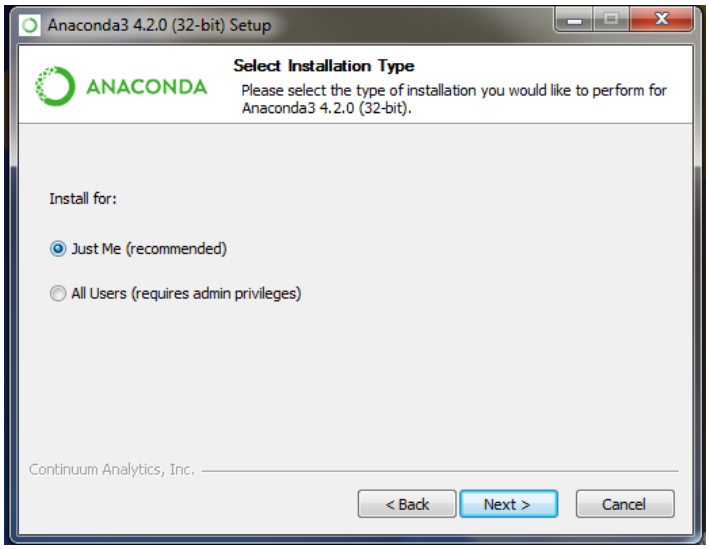
formaciononline.infosernt.com

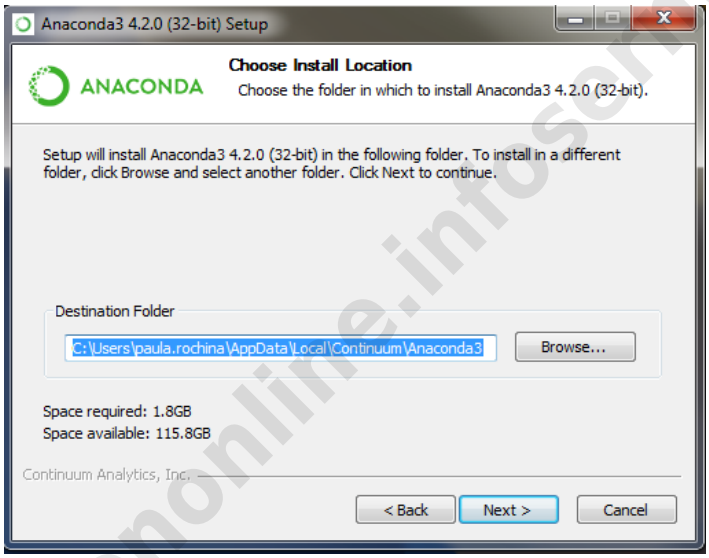
Para empezar a trabajar con Python en un entorno de **Big Data** y **Business Intelligence**, instalaremos una distribución gratuita llamada Anaconda, que pese a estar disponible solo en inglés, incluye multitud de utilidades que facilitarán el trabajo y es ideal para empezar en este entorno. Para instalarla simplemente tendremos que descargar la versión de Anaconda según el sistema operativo en el que estemos trabajando.

El proceso de instalación es automático, pero a continuación se muestran las diferentes ventanas que nos irán apareciendo en un entorno de Windows.

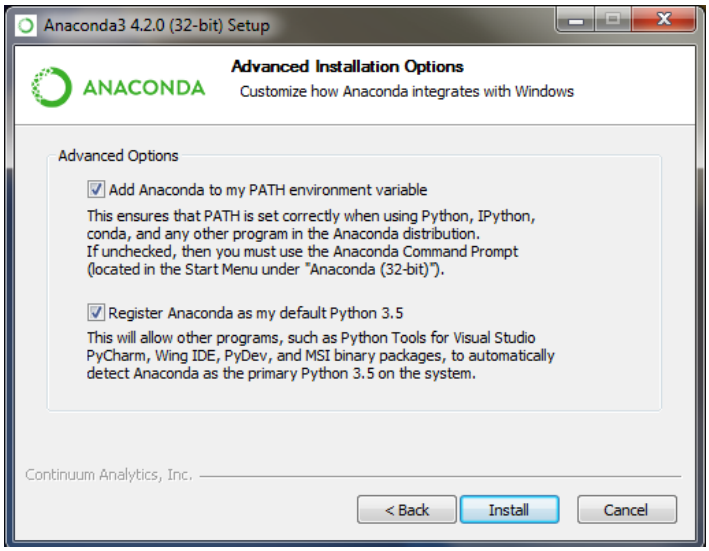
Seleccionamos la opción de instalación sólo para nuestro usuario:

formaciononline.infosernt.com



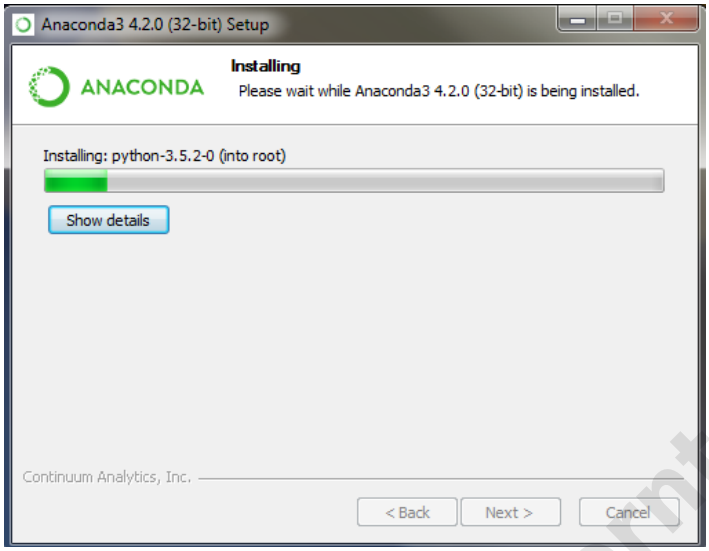



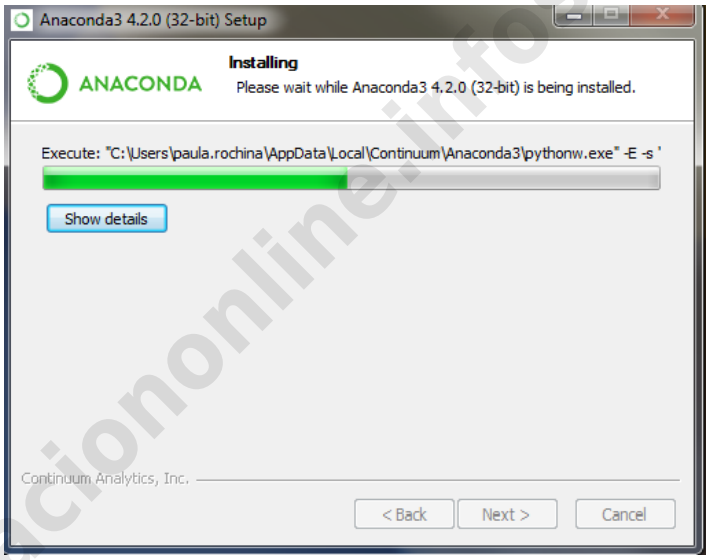

Dejamos las opciones marcadas por defecto, ya que son clave para que el sistema pueda invocar a **Python** cuando sea necesario:




formaciononline.infosernt.com

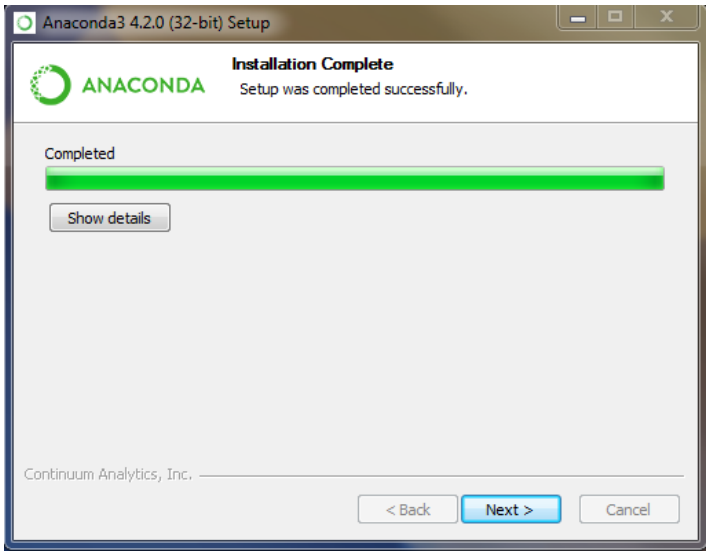
Comienza el proceso de instalación.

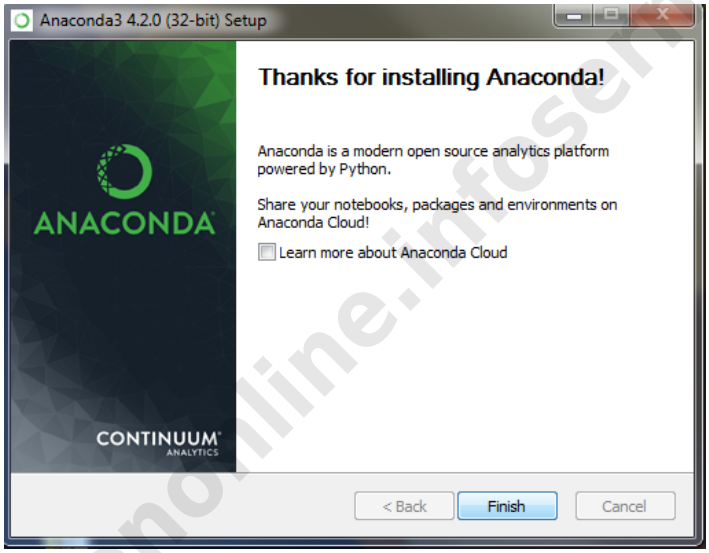



Finalizado el **proceso de instalación**, pulsamos **next**.

formaciononline.infosernt.com



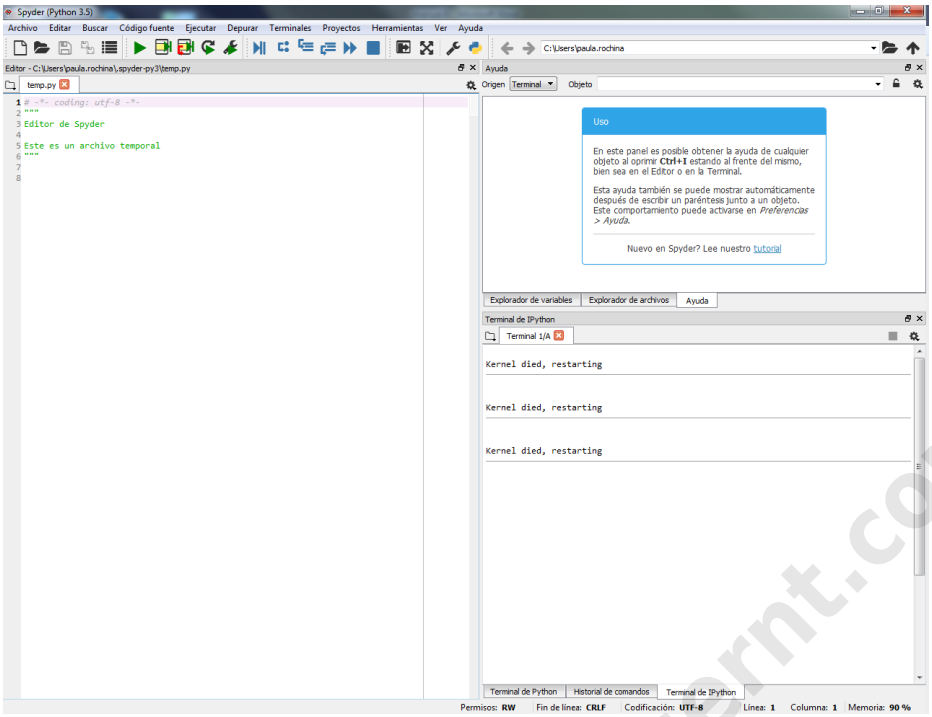




Tras estos sencillos pasos ya tendremos instalado la **versión de Anaconda** en nuestro ordenador.

## Utilidades instaladas

Spyder

formaciononline.infosernt.com



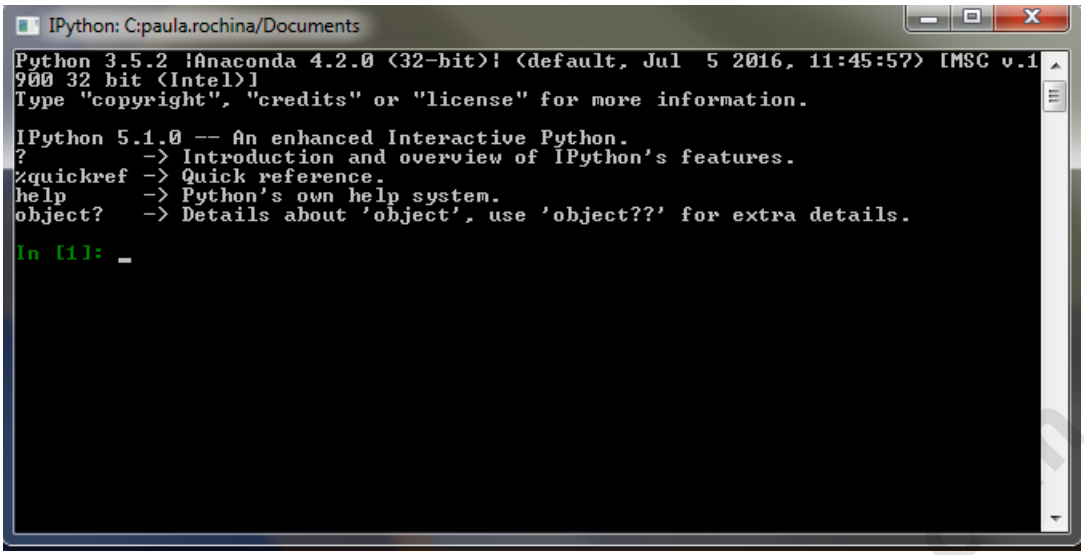

**Spyder** es un entorno de desarrollo en **Python**, orientado a científicos y con características similares a MATLAB:

- Consola con visualizador de variables.
- Visualizador de sys.path.
- Visualizador de variables de entorno.
- Funciones de dibujo de gráficos integrados.
- Autocompletado.
- Editor con resaltado de sintaxis
- Navegador de clases/funciones.
- Análisis de código.
- Autocompletado de código.

Está disponible tanto para Windows como para Linux.

IPython

formaciononline.infosernt.com




IPython es un Shell interactivo que añade funcionalidades extra al modo interactivo incluido con Python, como resaltado de líneas y errores mediante colores, una sintaxis adicional para el Shell, autocompletado mediante tabulador de variables, módulos y atributos; entre otras funcionalidades.

Actualmente multitud de paquetes utilizan IPython como librería o como intérprete interactivo. Es multiplataforma, software libre, y debido a la gran comunidad que tiene detrás, está en constante desarrollo y bien organizado, lo que lo hace extremadamente potente.

IPython comenzó en 2001 de la mano de Fernando Pérez, un investigador colombiano en Física de partículas. Lo importante de este programa es que nació en un ámbito científico de la mano de investigadores que demandaban un entorno más interactivo.

Funciones básicas de IPython

- **Autocompletado:** El autocompletado es una característica muy útil para no que tener que escribir un código y también, por ejemplo, para inspeccionar objetos rápidamente. Se activa con la tecla Tab. Funciona tanto con opciones instaladas como con variables definidas propias.
- **Historial:** El historial es una manera de recuperar resultados antiguos que no hemos guardado en una variable. Como las entradas en IPython están numeradas, no hay más que hacer referencia a la salida correspondiente: por ejemplo, si queremos recuperar el resultado obtenido en OUT[1], utilizaremos la variable _1.

formaciononline.infosernt.com

```
IPython: C:paula.rochina/Documents
Python 3.5.2 |Anaconda 4.2.0 (32-bit)| (default, Jul  5 2016, 11:45:57) [MSC v.1
900 32 bit (Intel)]
Type "copyright", "credits" or "license" for more information.

IPython 5.1.0 -- An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.

In [1]: 24 ** 2
Out[1]: 576

In [2]: _ #Esta variable almacena el útlimo resultado
Out[2]: 576

In [3]: _ * 3 # Lo multiplicamos por 3
Out[3]: 1728

In [4]: _1 # Recuperamos el resultado de 24 ** 2
Out[4]: 576

In [5]:
```

- **Ayuda:** La ayuda de IPython nos permite leer la documentación de objetos, funciones, etc., así como el código fuente donde se definieron. Se invoca utilizando el signo de interrogación ?.

Así pues, podemos llamar a la ayuda general de IPython como sigue:

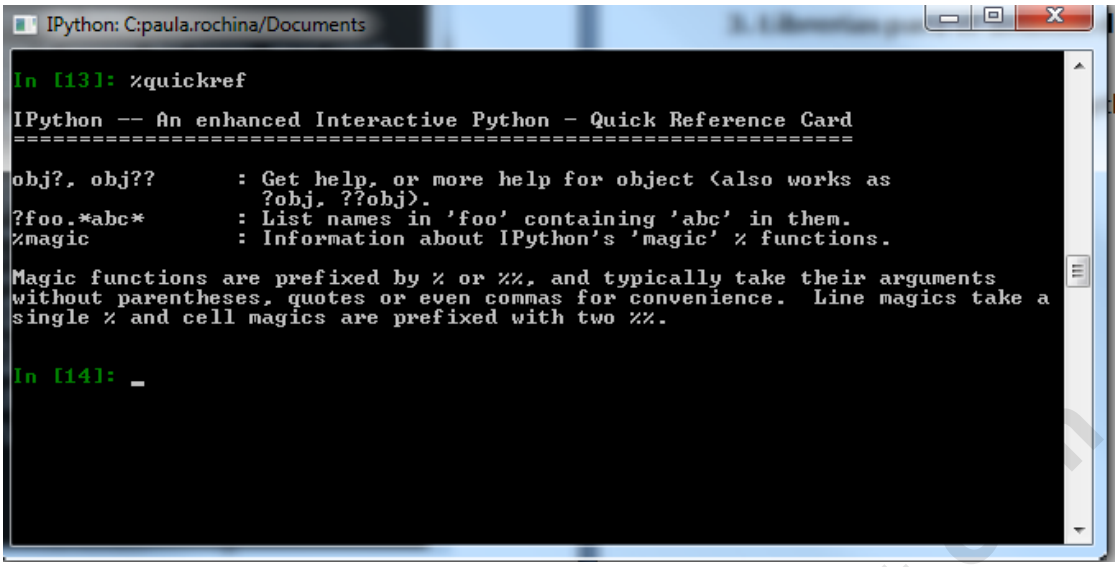
formaciononline.infosernt.com

```
IPython: C:paula.rochina/Documents

In [11]: ?

IPython -- An enhanced Interactive Python
=========================================

IPython offers a combination of convenient shell features, special commands
and a history mechanism for both input (command history) and output (results
caching, similar to Mathematica). It is intended to be a fully compatible
replacement for the standard Python interpreter, while offering vastly
improved functionality and flexibility.

At your system command line, type 'ipython -h' to see the command line
options available. This document only describes interactive features.

MAIN FEATURES
-------------

* Access to the standard Python help. As of Python 2.1, a help system is
  available with access to object docstrings and the Python manuals. Simply
  type 'help' (no quotes) to access it.

* Magic commands: type %magic for information on the magic subsystem.

* System command aliases, via the %alias command or the configuration file(s).

* Dynamic object information:

  Typing ?word or word? prints detailed information about an object.  If
  certain strings in the object are too long (docstrings, code, etc.) they get
  snipped in the center for brevity.

  Typing ??word or word?? gives access to the full information without
  snipping long strings. Long strings are sent to the screen through the less
  pager if longer than the screen, printed otherwise.

  The ?/?? system gives access to the full source code for any object (if
  available), shows function prototypes and other useful information.

  If you just want to see an object's docstring, type '%pdoc object' (without
  quotes, and without % if you have automagic on).

In [12]: _
```

También podemos pedir ayuda de cualquier función o método de **IPython** de forma similar, por ejemplo, si queremos obtener información de **'%magic'** lo podemos hacer como se muestra a continuación:

```
IPython: C:paula.rochina/Documents

In [12]: ?%magic
Docstring:
Print information about the magic function system.

Supported formats: -latex, -brief, -rest
File:      c:\users\paula.rochina\appdata\local\continuum\anaconda3\lib\site-pac
kages\ipython\core\magics\basic.py

In [13]: _
```

Otra forma de llamar a la ayuda rápida de IPython sería con el comando %quickref:




- **Guardar, editar, cargar:** Con IPython también podemos guardar, editar y ejecutar archivos. De esta forma, podemos aprovechar las ventajas del modo interactivo para llevar un registro de nuestros progresos.
    - El comando %edit lanza el editor de texto en consola por defecto.
    - La función %save recibe como argumentos el nombre del archivo que queremos y las líneas que queremos guardar en él.
    - El comando %run nos permitirá ejecutar o cargar archivos. No debemos olvidar ubicarnos en la carpeta que contiene dicho archivo.
- **Carpetas y consola:** Podemos navegar en las carpetas con el comando 'cd' de la siguiente forma:

formaciononline.infosernt.com

```
IPython: C:Documents/contenido máster URJC

In [14]: cd C:\Users\paula.rochina\Documents\contenido máster URJC
C:\Users\paula.rochina\Documents\contenido máster URJC

In [15]:
```

Sin embargo, la política de **IPython** es que el **lenguaje** esté primero, por lo cual 'cd' puede asignarse como nombre de una variable lo que provocará que no podamos usar 'cd' como comando.

```
IPython: C:Documents/contenido máster URJC

In [14]: cd C:\Users\paula.rochina\Documents\contenido máster URJC
C:\Users\paula.rochina\Documents\contenido máster URJC

In [15]: cd=1

In [16]: cd C:\Users\paula.rochina
  File "<ipython-input-16-63494000d20a>", line 1
    cd C:\Users\paula.rochina
       ^
SyntaxError: invalid syntax

In [17]:
```

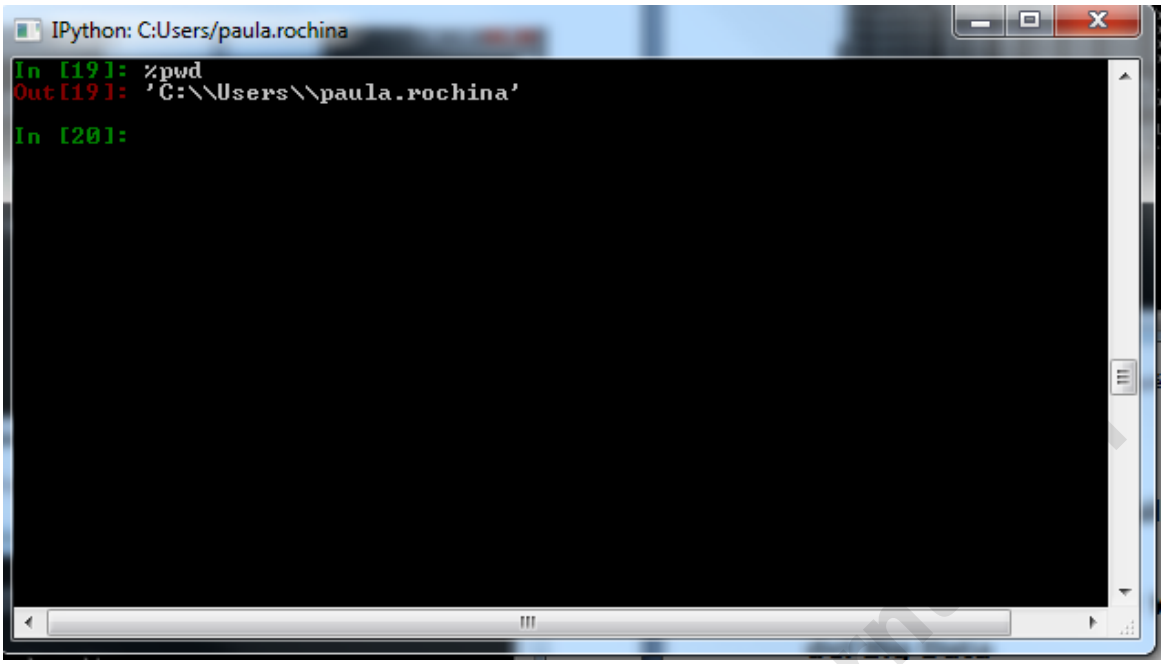
Para solucionar el problema anterior podemos usar el comando '%cd':

```
IPython: C:Users/paula.rochina

In [14]: cd C:\Users\paula.rochina\Documents\contenido máster URJC
C:\Users\paula.rochina\Documents\contenido máster URJC

In [15]: cd=1

In [16]: cd C:\Users\paula.rochina
  File "<ipython-input-16-63494000d20a>", line 1
    cd C:\Users\paula.rochina
       ^
SyntaxError: invalid syntax

In [17]: %cd C:\Users\paula.rochina
C:\Users\paula.rochina

In [18]:
```

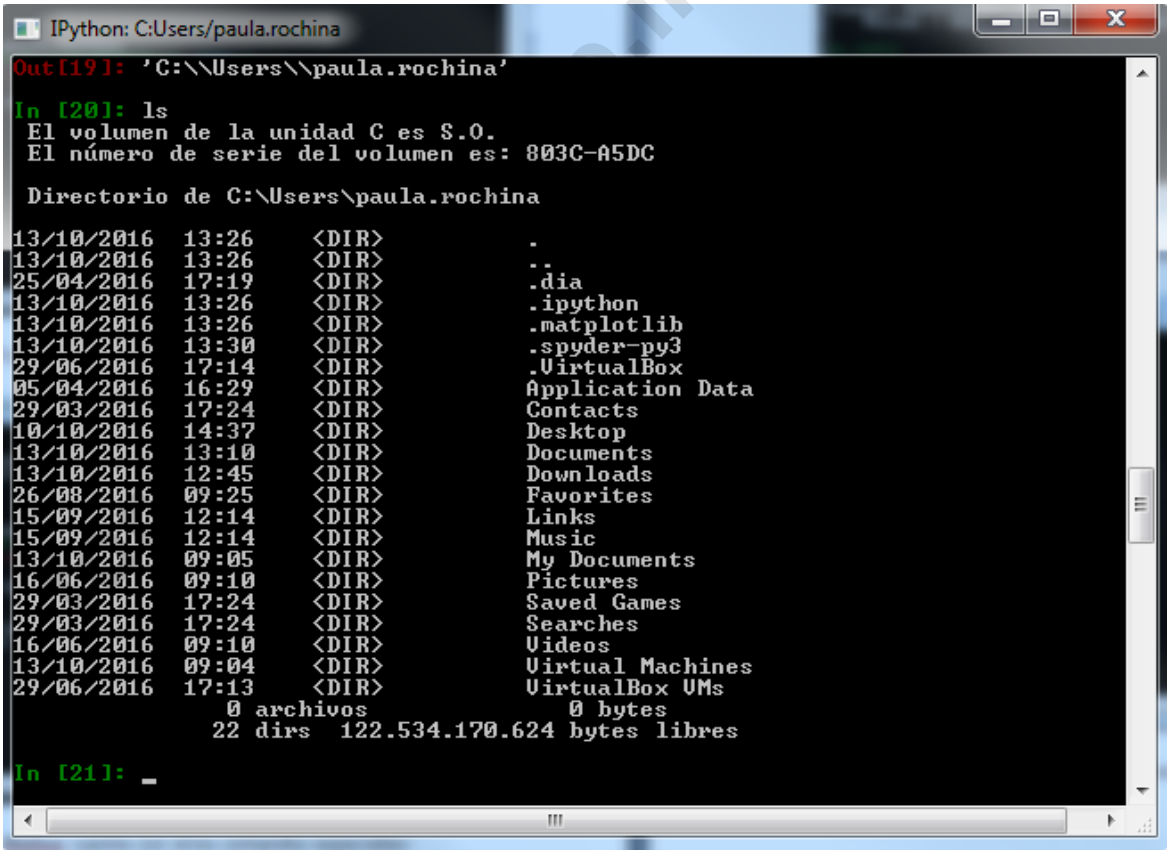
- **Otros comando:** IPython cuenta con otros comandos especiales:
    - ‘ **!ping** ’: Para hacer pruebas de red.

```
IPython: C:Users/paula.rochina

In [18]: !ping www.inesem.es

Haciendo ping a inesem.es [88.198.226.194] con 32 bytes de datos:
Respuesta desde 88.198.226.194: bytes=32 tiempo=43ms TTL=51
Respuesta desde 88.198.226.194: bytes=32 tiempo=42ms TTL=51
Respuesta desde 88.198.226.194: bytes=32 tiempo=43ms TTL=51
Respuesta desde 88.198.226.194: bytes=32 tiempo=44ms TTL=51

Estadísticas de ping para 88.198.226.194:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 42ms, Máximo = 44ms, Media = 43ms

In [19]:
```

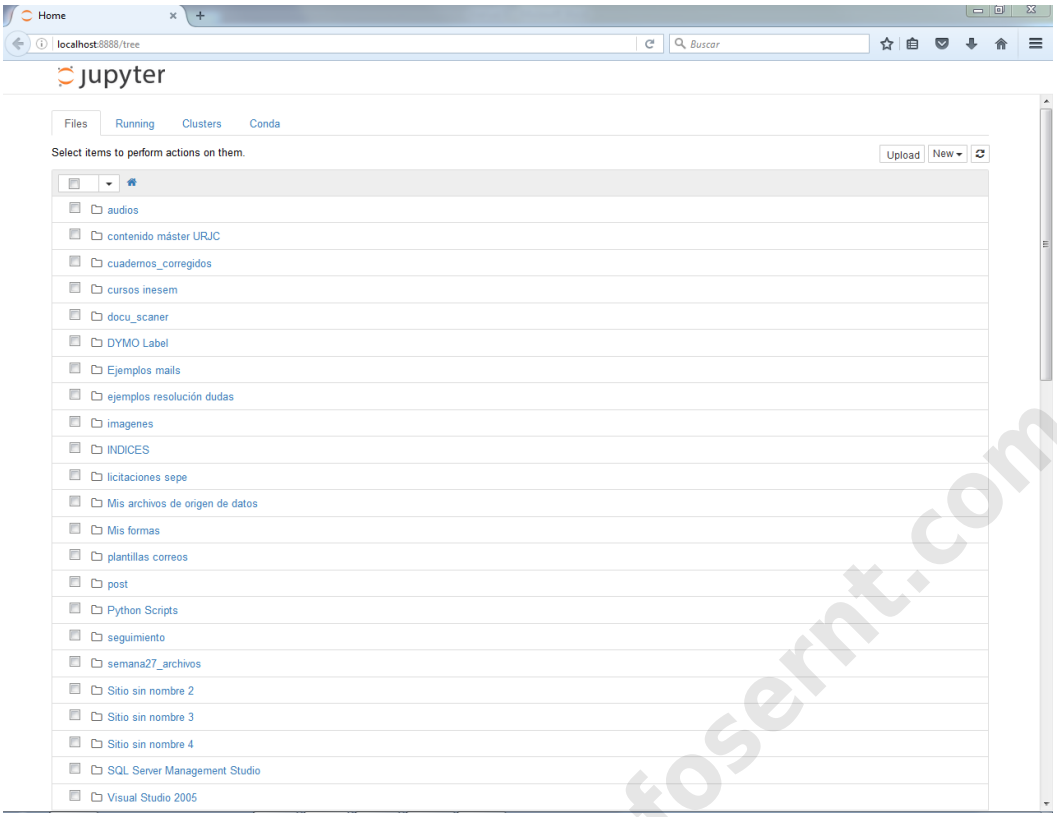
- ‘ **%pwd** ’ **:** Para saber en qué directorio nos encontramos actualmente.




- **ls:** Podemos saber que archivos y carpetas hay en la carpeta actual.



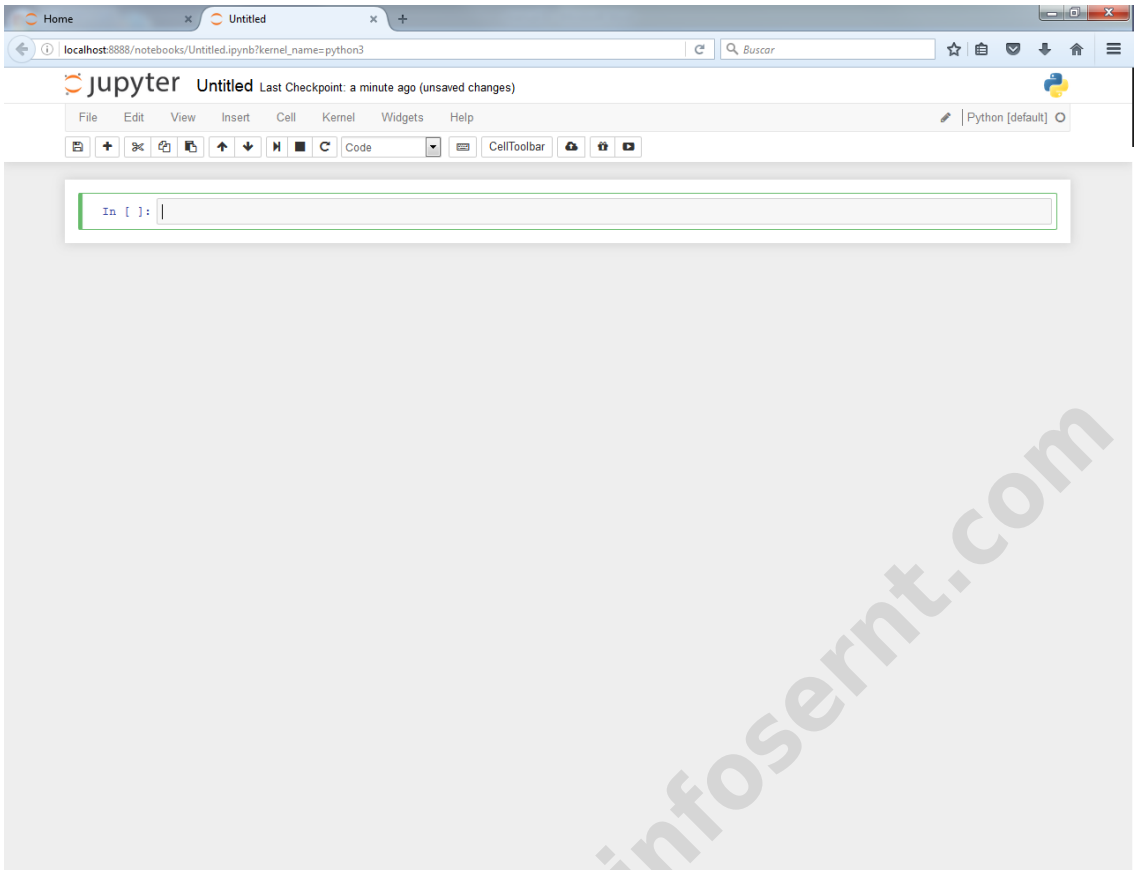

Jupyter Notebook




El proyecto Jupyter Notebook, antes conocido como **IPython Notebook**, proporciona una aplicación Web que nos permite crear y compartir documentos que contienen código vivo, ecuaciones, textos, visualizaciones, etc. Es decir, se trata de un entorno interactivo web de ejecución de código en el que, por ejemplo, se pueden incluir gráficas que ayuden en el análisis y explicación de los datos. Se utiliza para facilitar la explicación y reproducción de estudios y análisis.

Desde la página de inicio de Jupyter Notebook, mostrada en la imagen anterior, se pueden o bien crear cuadernos nuevos o abrir los existentes. Si hacemos clic en el botón **NEW** à **Notebooks** à **Python (default)**, veremos la página siguiente:

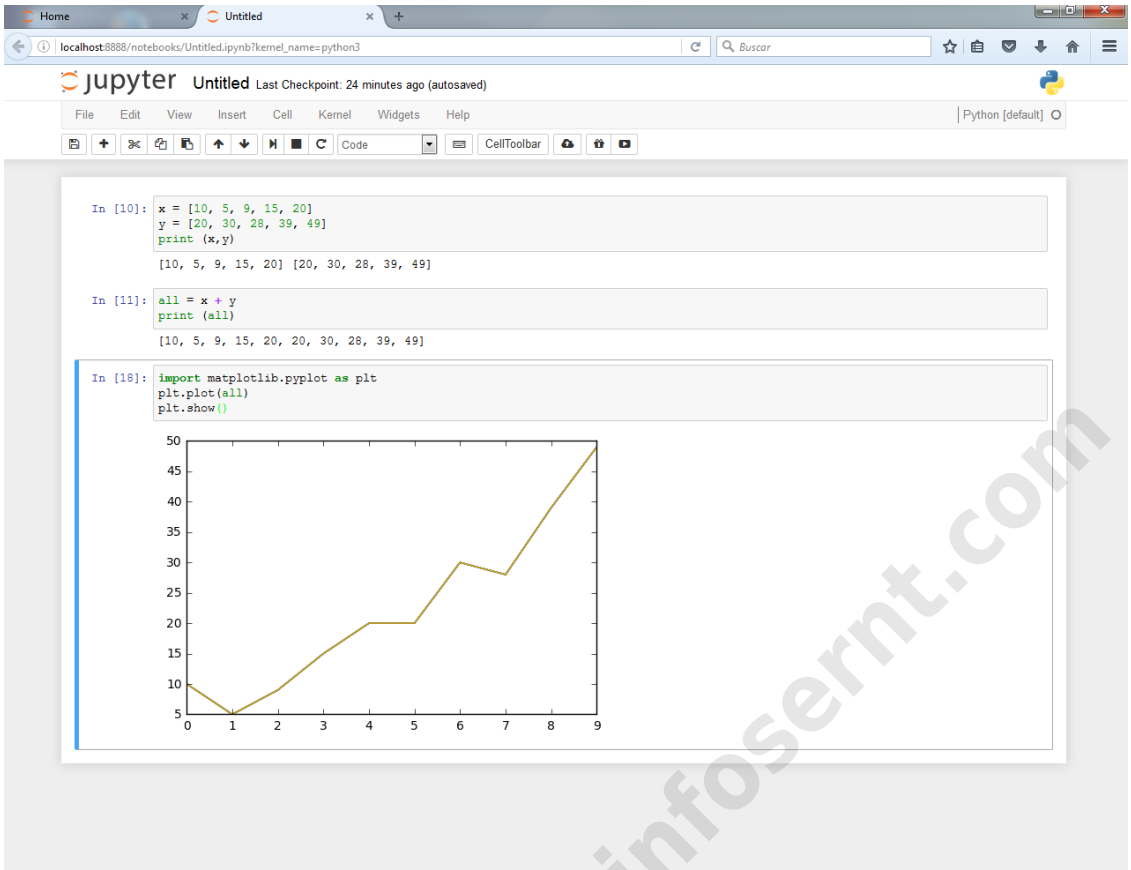
formaciononline.infosernt.com



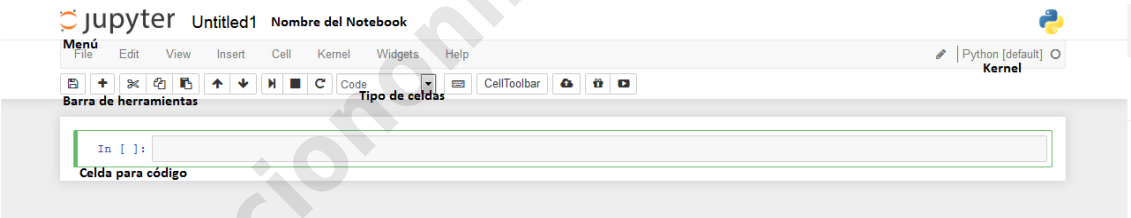

El área marcada con el mensaje In[]: es el área de entrada y ahí se puede escribir cualquier código Python válido, que se ejecutará cuando se presione Shift+Enter o se haga clic en el icono de Play (el triángulo que apunta a la derecha en la barra de herramientas).

Con la tecla Enter normal podremos escribir varías líneas, y con Alt+Enter, pasaremos a un nuevo bloque. El orden en que se ejecuten las líneas influye en los resultados. Se puede ejecutar todo de nuevo haciendo click en **Cell** à **Run All.** De esta forma todos los cálculos se reiniciarán, permitiendo de una forma muy cómoda hacer modificaciones en los datos iniciales y ejecutar de nuevo todo el proceso.

formaciononline.infosernt.com



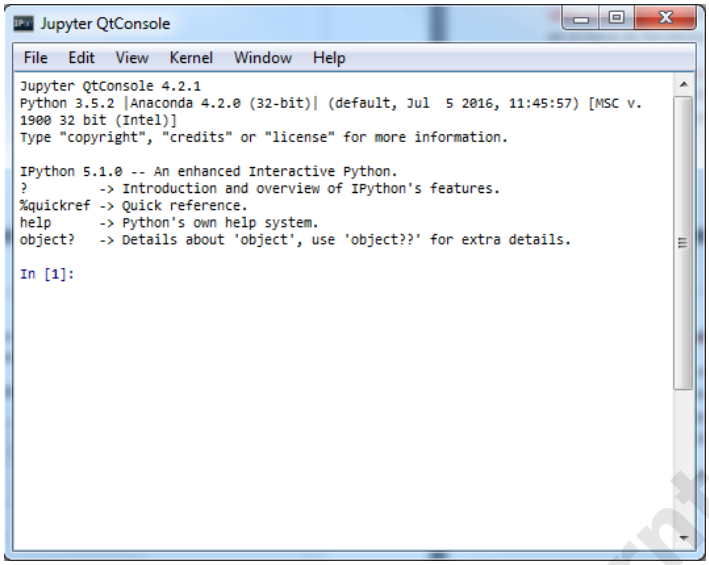

Interfaz de usuario de Jupyter Notebook




- **Nombre del Notebook:** Se puede cambiar el nombre haciendo click directamente sobre él. La extensión del archivo será .ypynb
- **Menú:** Es el menú de navegación en donde se tienen diferentes opciones para llevar a cabo con el Notebook.
- **Kernel:** Es el nombre del kernel con el que trabaja el notebook actual. Se puede cambiar el lenguaje del kernel desde la opción **Kernel** del Menú.
- **Barra de herramientas:** Contiene los iconos para las acciones más comunes.
- **Celda para código:** Es el espacio en donde se escribirá el código.

formaciononline.infosernt.com

Jupyter QTConsole



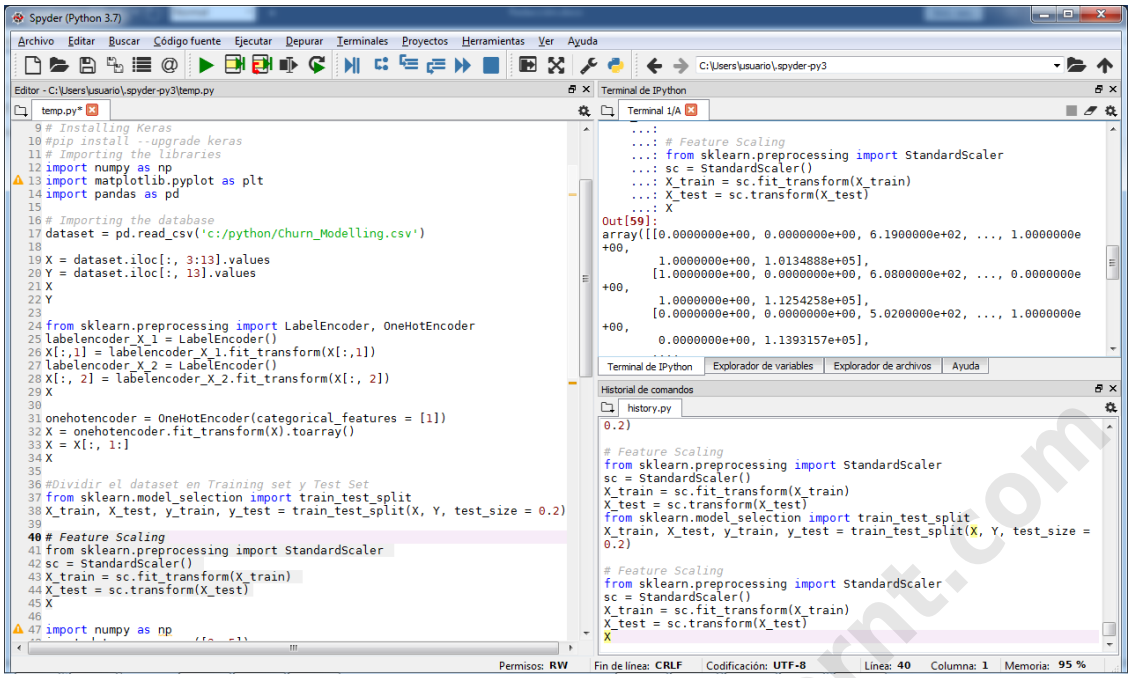

La **consola QT** es una **aplicación** muy ligera que se asemeja en gran medida a un terminal, pero ofrece una serie de mejoras en una única interfaz gráfica de usuario, tales como figuras en línea, edición de varias líneas con resaltado de sintaxis, gráficos, y mucho más.

La **consola QT** puede usarse con cualquier kernel de Jupyter. Una de las características más interesantes que ofrece la consola QT es, como ya hemos hablado, la posibilidad de incrustar figuras en línea, éstas pueden representarse con matploit en IPython, o la biblioteca plotting elegida en el kernel seleccionado.

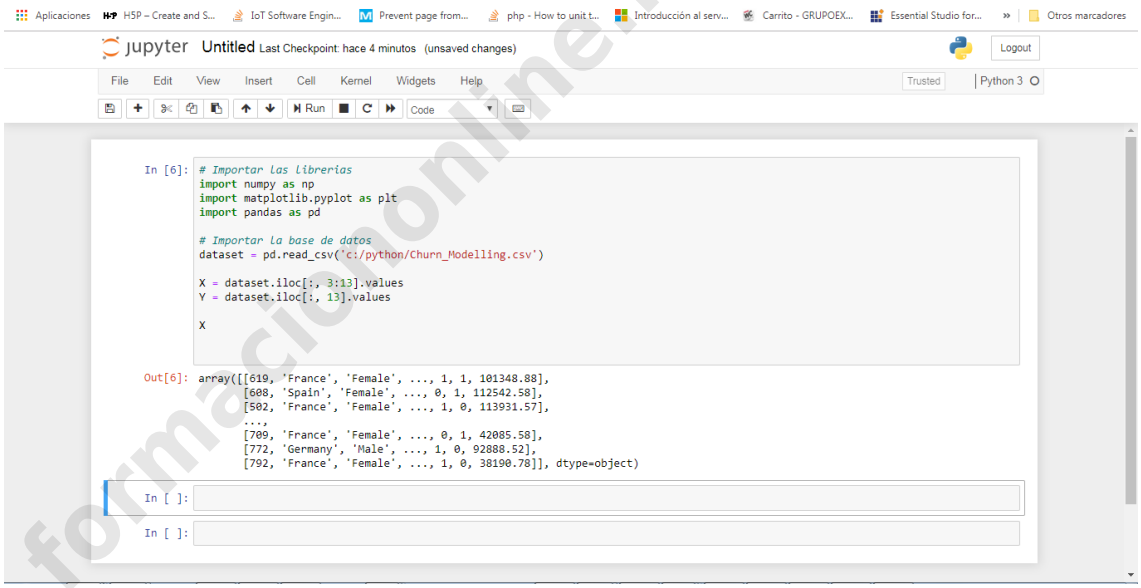
Prueba de código del curso

Para probar nuestro código, podemos utilizar **Spyder**.

formaciononline.infosernt.com




Pero se puede utilizar cualquier otro método para lanzar código Python, como por ejemplo utilizar un notebook de Jupyter:




También es posible utilizar la consola, o algún IDE (entorno de desarrollo integrado) como **PyCharm**.

**Para empezar a trabajar con Python en un entorno de Big Data y Business Intelligence, instalaremos una distribución gratuita llamada Anaconda, que pese a estar disponible solo en inglés, incluye multitud de utilidades que facilitarán el**

formaciononline.infosernt.com

**trabajo y es ideal para empezar en este entorno. Para instalarla simplemente tendremos que descargar la versión de Anaconda según el sistema operativo en el que estemos trabajando.**

Verdadero.

Falso.

formaciononline.infosernt.com

# Librería de Python para Machine Learning.

Una de las grandes ventajas que ofrece **Python** sobre otros lenguajes de programación, es lo grande y prolifera que es la comunidad de desarrolladores que lo rodean; esta comunidad ha contribuido con una gran variedad de librerías de primer nivel que extienden las funcionalidades del lenguaje.

Gracias a una serie de librerías que veremos a lo largo del presente punto, Python se utiliza habitualmente para el cálculo estadístico y la visualización gráfica de los datos para extraer conclusiones y tomar decisiones. Pero lo que hace interesante a Python es que no simplemente un lenguaje que permite visualizar datos, sino que también tiene una gran capacidad para automatizar procesos, extraer datos o usarlo para aprendizaje automático. Con Python, podremos cambiar grandes grupos de datos a través de programación sencilla y simplificar la utilización de las APIs para capturar datos y extraer información.

## 2.1. Cálculos matemáticos y estadísticos con Numpy y Pandas

Los cálculos matemáticos y estadísticos están íntimamente relacionados con el análisis de datos y la toma de decisiones en Businees Intelligence, calcular el comportamiento de los datos y extraer estadísticas de su evolución nos serán de utilidad como analistas de datos.

En Python, existen varias librerías que nos pueden ayudar en estos cálculos, dos de ellas son **Numpy** y **Pandas**, cada una de las cuales está orientada a trabajar con los datos de una manera concreta a través de las funciones que implementan y que definiremos en mayor profundidad a lo largo del presente punto.

**Numpy**

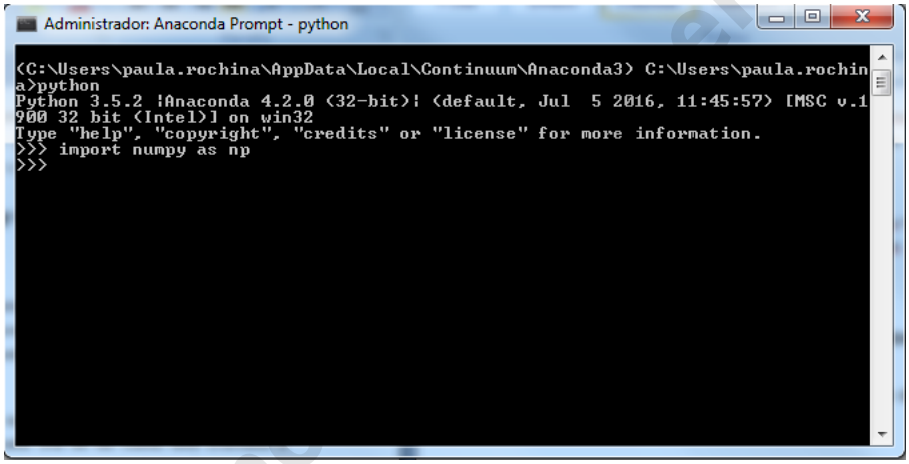
Como cualquier otra librería de Python, cuenta con un sitio oficial, a través del cual se generan actualizaciones y documentación, así como tutoriales relacionados con el uso de las funciones que implementa este paquete.

Numpy, está pensada fundamentalmente para a computación científica, por lo que implementa funciones para el manejo de vectores y matrices, así como de algebra linear. Otra característica resaltable de este módulo, es que permite incorporar código fuente de otros lenguajes programación como C/C++ o Fortran, permitiendo un incremento en su compatibilidad e implementación.

formaciononline.infosernt.com

Antes de comenzar a trabajar con las funcionalidades ofrecidas por esta librería, debemos comprobar si ya la tenemos instalada dentro de nuestro paquete de Python en el entorno de desarrollo que estamos utilizando, o si por el contrario debemos descargárnosla y proceder a su instalación e integración. En Anaconda, el entorno de desarrollo con el que estamos trabajando en la presente unidad didáctica, a partir de su versión 4.2.0 ya se incluye por defecto alrededor de 100 de paquetes relacionados con el testeo y análisis de datos en Python, entre los que se encuentra la librería Numpy, por tanto, si nuestra versión se corresponde con esta o alguna posterior ya lo tendremos instalado.

No obstante, para comprobar si tenemos Numpy integrado basta con abrir un terminal (Anaconda Prompt) y ejecutar lo siguiente:




Si tras la ejecución de la línea "**import numpy as np**" no nos aparece ningún error en el terminal es porque tenemos la librería instalada, en caso contrario debemos proceder a su instalación. Por otro lado, cuando importamos la librería para usar sus funciones dentro de nuestro código, nótese que creamos una especie de "alias" y la renombramos como np (notación que generalmente se usa por convenio), con la finalidad de no tener que escribir el nombre completo de la librería cada vez que utilizamos algunas de las funciones que implementa.

Otra forma de comprobar si tenemos no solo esta librería sino cualquier otro paquete de nuestro interés, instalado para Python en nuestra versión de Anaconda, sería ejecutar en una consola de comandos la línea siguiente:

**conda list**

Esto nos devuelve una lista con todos los paquetes y sus versiones que tenemos instalados. A continuación, se muestra la ejecución de este comando en un terminal de Prompt de Anaconda.

```
Administrador: C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Reservados todos los derechos.

C:\Users\paula.rochina>conda list
# packages in environment at C:\Users\paula.rochina\AppData\Local\Continuum\Anac
onda3:
#
_license                  1.1                      py35_1
_nb_ext_conf              0.3.0                    py35_0
alabaster                 0.7.9                    py35_0
anaconda                  4.2.0                np111py35_0
anaconda-clean            1.0.0                    py35_0
anaconda-client           1.5.1                    py35_0
anaconda-navigator        1.3.1                    py35_0
argcomplete               1.0.0                    py35_1
astroid                   1.4.7                    py35_0
astropy                   1.2.1                np111py35_0
babel                     2.3.4                    py35_0
backports                 1.0                      py35_0
beautifulsoup4            4.5.1                    py35_0
bitarray                  0.8.1                    py35_1
blaze                     0.10.1                   py35_0
bokeh                     0.12.2                   py35_0
boto                      2.42.0                   py35_0
bottleneck                1.1.0                np111py35_0
bzip2                     1.0.6                    vc14_3  [vc14]
cffi                      1.7.0                    py35_0
chest                     0.2.3                    py35_0
click                     6.6                      py35_0
cloudpickle               0.2.1                    py35_0
clyent                    1.2.2                    py35_0
colorama                  0.3.7                    py35_0
comtypes                  1.1.2                    py35_0
conda                     4.2.9                    py35_0
conda-build               2.0.2                    py35_0
configobj                 5.0.6                    py35_0
console_shortcut          0.1.1                    py35_1
contextlib2               0.5.3                    py35_0
cryptography              1.5                      py35_0
curl                      7.49.0                   vc14_0  [vc14]
cycler                    0.10.0                   py35_0
cython                    0.24.1                   py35_0
cytoolz                   0.8.0                    py35_0
dask                      0.11.0                   py35_0
datashape                 0.5.2                    py35_0
decorator                 4.0.10                   py35_0
dill                      0.2.5                    py35_0
docutils                  0.12                     py35_2
dynd-python               0.7.2                    py35_0
entrypoints               0.2.2                    py35_0
et_xmlfile                1.0.1                    py35_0
fastcache                 1.0.2                    py35_1
filelock                  2.0.6                    py35_0
flask                     0.11.1                   py35_0
flask-cors                2.1.2                    py35_0
```

Como se aprecia en la imagen anterior, nos aparecen por orden alfabético todos los paquetes que tenemos instalados.

Por último, también podríamos comprobar si tenemos instalado un paquete concreto si ejecutamos en la consola de comandos la instrucción

**conda search <nombre del paquete>**

Lo que nos dará como resultado información referente al paquete que estamos buscando en el caso

de que lo tengamos instalado.

```
Administrador: C:\Windows\system32\cmd.exe
C:\Users\paula.rochina>conda search numpy
Fetching package metadata .........
numpy                     1.6.2                    py26_0  defaults
                          1.6.2                    py27_0  defaults
                          1.6.2                    py26_4  defaults
                          1.6.2                   py26_p4  defaults        [m
kl]
                          1.6.2                    py27_4  defaults
                          1.6.2                   py27_p4  defaults        [m
kl]
                          1.6.2                   py26_p5  defaults        [m
kl]
                          1.6.2                   py27_p5  defaults        [m
kl]
                          1.7.0                    py26_0  defaults
                          1.7.0                    py27_0  defaults
                          1.7.0                    py33_0  defaults
                          1.7.1                    py26_0  defaults
                          1.7.1                   py26_p0  defaults        [m
kl]
                          1.7.1                    py27_0  defaults
                          1.7.1                   py27_p0  defaults        [m
kl]
                          1.7.1                    py33_0  defaults
                          1.7.1                   py33_p0  defaults        [m
kl]
                          1.7.1                    py26_1  defaults
                          1.7.1                   py26_p1  defaults        [m
kl]
                          1.7.1                    py27_1  defaults
                          1.7.1                   py27_p1  defaults        [m
kl]
                          1.7.1                    py33_1  defaults
                          1.7.1                   py33_p1  defaults        [m
kl]
                          1.7.1                    py26_2  defaults
                          1.7.1                   py26_p2  defaults        [m
kl]
                          1.7.1                    py27_2  defaults
                          1.7.1                   py27_p2  defaults        [m
kl]
                          1.7.1                    py33_2  defaults
                          1.7.1                   py33_p2  defaults        [m
kl]
                          1.7.1                    py26_3  defaults
                          1.7.1                   py26_p3  defaults        [m
kl]
                          1.7.1                    py27_3  defaults
                          1.7.1                   py27_p3  defaults        [m
kl]
                          1.7.1                    py33_3  defaults
                          1.7.1                   py33_p3  defaults        [m
kl]
                          1.8.0                    py26_0  defaults
                          1.8.0                   py26_p0  defaults        [m
```

Si, por el contrario, no tenemos el paquete necesario instalado podemos desde un mismo terminal proceder a su instalación mediante la ejecución de la línea siguiente:

conda install <nombre del paquete>

```
Administrador: C:\Windows\system32\cmd.exe - conda  install numpy
Solving package specifications: ..........

Package plan for installation in environment C:\Users\paula.rochina\AppData\Loca
l\Continuum\Anaconda3:

The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    conda-env-2.6.0            |                0          488 B
    numpy-1.11.2               |            py35_0        3.1 MB
    conda-4.2.12               |            py35_0        441 KB
    ------------------------------------------------------------
                                           Total:        3.5 MB

The following NEW packages will be INSTALLED:

    conda-env: 2.6.0-0

The following packages will be UPDATED:

    conda:     4.2.9-py35_0  --> 4.2.12-py35_0
    numpy:     1.11.1-py35_1 --> 1.11.2-py35_0

Proceed ([y]/n)?
```

Igualmente, como se observa en la imagen anterior, cuando intentamos instalar un paquete que ya existe, se mostrará información de este paquete, así como la posibilidad de actualizarlo en el caso de que se haya encontrado una versión más reciente.

**Creando arrays con Numpy**

Para la creación de arrays a partir de Numpy podemos optar por cuatro formas distintas:

- A partir de secuencias de datos (listas o tuplas) de Python.

- Haciendo uso de funciones propias de Numpy.

- Leyendo los datos desde un fichero.

- Copiando otro array.

La principal finalidad del módulo Numpy es la creación y modificación de arrays multidimensionales, lo que se consigue a través de los métodos y funciones de la clase *ndarray* que implementa. A continuación, mostramos un resumen de algunas de las funciones de esta clase.

| Atributo | Descripción |
|---|---|
| ndim | Devuelve el número de dimensiones del array. |
| shape | Devuelve la dimensión del array, es decir, una tupla de enteros indicando el tamaño del array en cada dimensión. |
| size | Proporciona el número total de elementos del array. |

| | |
|---|---|
| dtype | Es un objeto que describe el tipo de elementos del array. |
| itemsize | Devuelve el tamaño en bytes del array. |
| data | El buffer contiene los elementos actuales del array. |

Entre las funciones para crear arrays, y dependiendo de las necesidades que demande el problema a resolver tenemos las siguientes funciones implementadas:

| Atributo | Descripción |
|---|---|
| identity(n,dtype) | Crea una matriz identidad, esto es, una matriz cuadrada con unos en la diagonal, donde n es el tamaño de la matriz y dtype el tipo de dato que contiene |
| ones(shape,dtype) | Crea un array de 1s compuesto de shape elementos. |
| zeros(shape,dtype) | Crea un array de 0s compuesto de shape elementos. |
| empty(shape,dtype) | Crea un array de shape elementos sin entrada. |
| eye(N,M,k,dtype) | Crea un array bidimensional con 1s en la diagonal k y 0s en el resto. Es similar a identity. Los argumentos son opcionales, siendo N el número de filas, M el número de columnas y k el índice de la diagonal. Cuando k=0 nos referimos a la diagonal principal. |
| arange([start,]stop[step,],dtype=None) | Crea un array con un rango de valores a partir del valor inicial start y hasta el valor final stop, incrementando según el valor almacenado en la variable step. Si no se establece step, python creará un valor por defecto. |
| linspace(start,stop,num,endpoint=True,retstep=False) | Crea un array con valor inicial start, valor final stop y num elementos. |
| meshgrid(x,y) | Genera una matriz de coordenadas a partor de dos arrays x e y. |

Mostramos a continuación algunos ejemplos de uso de las funciones anteriores.

formaciononline.infosernt.com

Administrador: Anaconda Prompt - python

```
900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy as np # Importamos numpy como el alias np
>>> array_prueba = np.arange (10) #Creamos un array de 0 a 9 separados de uno en
 uno
>>> array_prueba #Mostramos por pantalla el array creado
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> type (array_prueba) #Comprobamos que es un ndarray
<class 'numpy.ndarray'>
>>> array_prueba.ndim #Consultamos el número de dimensiones
1
>>> array_prueba.shape #Consultamos la dimensión
(10,)
>>> array_prueba.size #Consultamos el número total de elementos
10
>>> array_prueba.dtype #Consultamos el tipo de elementos que contiene el array
dtype('int32')
>>> array_prueba.itemsize #tamaño en bytes
4
>>> array_prueba.data #consultamos el buffer de memoria para el array
<memory at 0x03BAA4A8>
>>>
```

Administrador: Anaconda Prompt - python

```
(C:\Users\paula.rochina\AppData\Local\Continuum\Anaconda3) C:\Users\paula.rochin
a>python
Python 3.5.2 |Anaconda 4.2.0 (32-bit)| (default, Jul  5 2016, 11:45:57) [MSC v.1
900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from numpy import * #Importamos todo el módulo numpy
>>> zeros ( (3,4)) #Creamos un array nulo de 3 filas y 4 columnas
array([[ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.]])
>>> ones (9) #Creamos un array unitario de 1 fila y 9 columnas
array([ 1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.])
>>> empty ( (2,3) ) #creamos un array sin entradas de 2x3
array([[  2.91419137e-317,   9.69912776e-291,   9.55762261e-291],
       [  8.65699573e-315,   6.64372050e-318,   3.41779567e+098]])
>>> arange ( 10, 30, 5) # Creamos un array con inicio en 10 final en 30 con incr
emento de 5 en 5
array([10, 15, 20, 25])
>>> linspace( 0, 2, 9) #array de 9 elementos de 0 a 2
array([ 0.  ,  0.25,  0.5 ,  0.75,  1.  ,  1.25,  1.5 ,  1.75,  2.  ])
>>> _
```

**Creando matrices con Numpy**

Los objetos matriz son un subtipo de array bidimensional de Numpy (ndarray), y por tanto heredan todos los atributos y métodos de ndarry. La mayor ventaja que presentan las matrices es que implementan la multiplicación entre este tipo de elementos, que reemplaza a la multiplicación de elemento a elemento de los arrays. Un ejemplo de esta diferencia en el producto, lo podemos ver a continuación:

```
IPython: C:paula.rochina/Documents
In [1]: from numpy import *

In [2]: x = array ( ((2,3), (3,5)) )

In [3]: y = array ( ((1,2), (5,1)) )

In [4]: x * y
Out[4]:
array([[ 2,  6],
       [15,  5]])

In [5]: x = matrix ( ((2,3), (3,5) )
   ...: )

In [6]: y = matrix ( ((1,2), (5,1)) )

In [7]: x * y
Out[7]:
matrix([[17,  7],
        [28, 11]])

In [8]: _
```

Como se observa en la imagen anterior, el resultado de multiplicar una matriz definida como un array bidimensional o como un objeto matriz es diferente, esto es debido a lo que explicábamos unas líneas más arriba, la forma en que se implementa el producto en cada una de las clases es diferente. Mientras que el producto de arrays multiplica uno a uno los elementos, en matrix el producto es el definido en el álgebra lineal para matrices.

A parte del producto, dentro de la clase matrix, podemos hacer uso de todas las funciones del álgebra lineal como la transposición, la diagonalización, etc. Veamos algunos ejemplos:

```
IPython: C:paula.rochina/Documents
In [1]: from numpy import matrix

In [2]: a = matrix([[1,3,-5],[3,4,2],[-5,2,0]])

In [3]: a.T #Calculamos la transpuesta de a
Out[3]:
matrix([[ 1,  3, -5],
        [ 3,  4,  2],
        [-5,  2,  0]])

In [4]: a.H #calculamos la transpuesta y conjugada de a
Out[4]:
matrix([[ 1,  3, -5],
        [ 3,  4,  2],
        [-5,  2,  0]])

In [5]: b = a.I #calculamos la inversa de a

In [6]: a
Out[6]:
matrix([[ 1,  3, -5],
        [ 3,  4,  2],
        [-5,  2,  0]])

In [7]: b
Out[7]:
matrix([[ 0.02439024,  0.06097561, -0.15853659],
        [ 0.06097561,  0.15243902,  0.10365854],
        [-0.15853659,  0.10365854,  0.0304878 ]])

In [8]: a*b #multiplicamos ambas matrices
Out[8]:
matrix([[  1.00000000e+00,   1.11022302e-16,   5.55111512e-17],
        [  0.00000000e+00,   1.00000000e+00,   5.55111512e-17],
        [  0.00000000e+00,   0.00000000e+00,   1.00000000e+00]])

In [9]:
```

Con el paquete *linalg* podemos calcular el determinante (det), resolver ecuaciones lineales (solve) y calcular autovalores y autovectores.

```
IPython: C:paula.rochina/Documents
In [15]: from numpy import *

In [16]: a = matrix ([[2,3,5], [-1,4,9],[7,-3,-5]])

In [17]: linalg.det(a) #calculamos el determinante de a
Out[17]: 63.00000000000005

In [18]: c = linalg.eig(a) #cálculo de autovalores y autovectores

In [19]: c[0]
Out[19]:
array([ 6.56956917+0.j        , -2.78478459+1.35449059j,
       -2.78478459-1.35449059j])

In [20]: c[1]
Out[20]:
matrix([[ 0.71450600+0.j        , -0.13363473+0.07757244j,
         -0.13363473-0.07757244j],
        [ 0.64777231+0.j        , -0.79106222+0.j        , -0.79106222-0.j
  ],
        [ 0.26433353+0.j        ,  0.58150578-0.11043488j,
          0.58150578+0.11043488j]])

In [21]:
```

Entre los recursos del módulo dispone de un enlace a una lista de todas las funciones del algebra lineal implementadas en el paquete linalg.

**Polinomios en Numpy**

Como hemos visto hasta ahora Numpy nos permite hacer cálculos de álgebra lineal, para lo que dispone también de una clase para trabajar con polinomios (numpy.polynomial). Entre los recursos del módulo encontrará un enlace a todos los objetos más usuales definidos en esta clase. Mostramos a continuación algunos ejemplos de uso.

```
IPython: C:paula.rochina/Documents

In [34]: from numpy.polynomial import Polynomial as P

In [35]: polinomio = P([1,2,3]) #Declaramos el polinomio p(x)=1 + 2*x + 3*(x)^2
    ...:

In [36]: print(polinomio)
poly([ 1.  2.  3.])

In [37]: mul = polinomio * polinomio #multiplicacion de polinomios

In [38]: print(mul)
poly([  1.   4.  10.  12.   9.])

In [39]: potencia = mul**3 #Elevamos el polinomio al cubo

In [40]: print(potencia)
poly([  1.00000000e+00   1.20000000e+01   7.80000000e+01   3.40000000e+02
   1.09500000e+03   2.71200000e+03   5.28400000e+03   8.13600000e+03
   9.85500000e+03   9.18000000e+03   6.31800000e+03   2.91600000e+03
   7.29000000e+02])

In [41]: eva = polinomio(2) #Evaluamos el polinomio en 2

In [42]: print(eva)
17.0

In [43]: _
```

**Concepto de Broadcasting**

Broadcasting es un mecanismo muy potente que permite a numpy trabajar con arrays de diferentes dimensiones. Así, si los objetos con los que estamos operando no tienen las mismas dimensiones, Numpy puede adaptar algunas de ellas para completar la operación.

Para llevar a cabo este mecanismo, Numpy alinea los arrays a la derecha y empieza a comprar las dimensiones por el final. Si son todas compatibles realiza la operación, y si no lanza un error. Entendemos que dos dimensiones son compatibles si se cumple una de las dos condiciones

siguientes:

- Son iguales, 0.
- Una de ellas es 1.

Si los objetos no tienen las mismas dimensiones, pero son compatibles, se asume que las restantes son 1 en el array de menor dimensión. Vamos a ver esto mejor con algunos ejemplos.

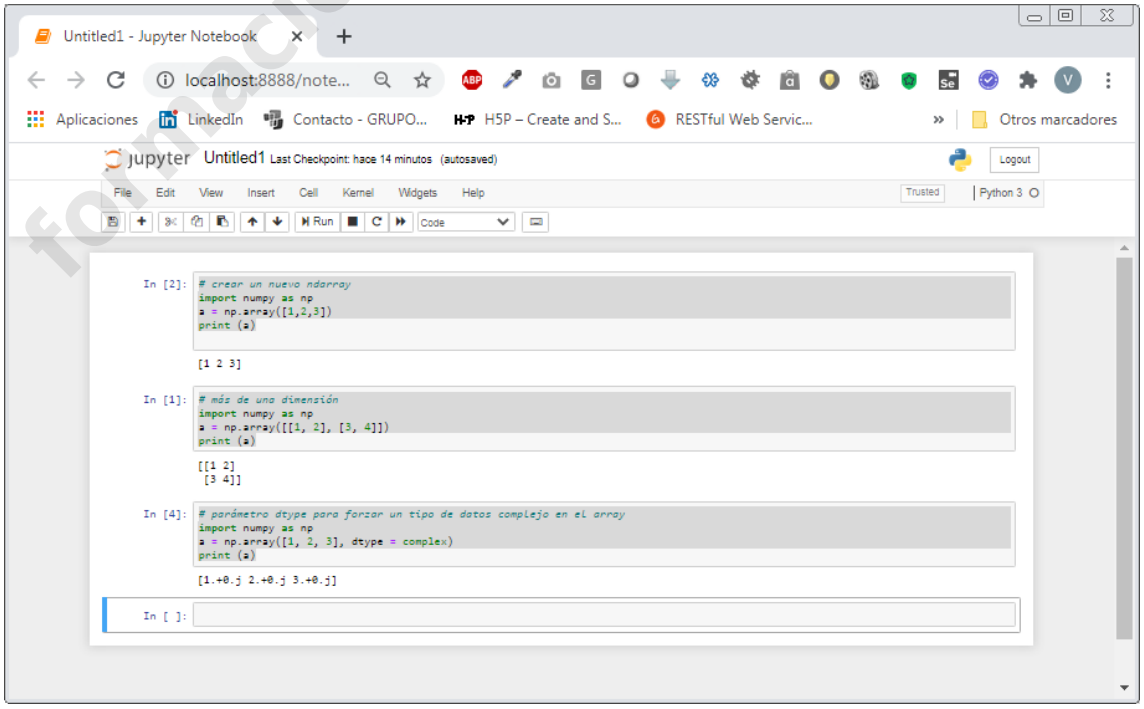
```
IPython: C:paula.rochina/Documents
In [43]: import numpy as np

In [44]: x = np.zeros((3,3))

In [45]: x
Out[45]:
array([[ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.]])

In [46]: x.shape #dimensiones de x
Out[46]: (3, 3)

In [47]: y = np.array([1,2,3])

In [48]: y
Out[48]: array([1, 2, 3])

In [49]: y.shape #Dimensiones de y
Out[49]: (3,)

In [50]: x + y #Se expande la primera dimensión del menor (y)
Out[50]:
array([[ 1.,  2.,  3.],
       [ 1.,  2.,  3.],
       [ 1.,  2.,  3.]])

In [51]: x2 = np.arange(4)

In [52]: y2 = np.arange(4).reshape((4,1))

In [53]: x2.shape
Out[53]: (4,)

In [54]: y2.shape
Out[54]: (4, 1)

In [55]: x2
Out[55]: array([0, 1, 2, 3])

In [56]: y2
Out[56]:
array([[0],
       [1],
       [2],
       [3]])

In [57]: x2+y2*1j #Se expanden la segunda dimensión de y más la primera de x
Out[57]:
array([[ 0.+0.j,  1.+0.j,  2.+0.j,  3.+0.j],
       [ 0.+1.j,  1.+1.j,  2.+1.j,  3.+1.j],
       [ 0.+2.j,  1.+2.j,  2.+2.j,  3.+2.j],
       [ 0.+3.j,  1.+3.j,  2.+3.j,  3.+3.j]])

In [58]: _
```

**Ejemplos de código con Numpy**

A continuación, se muestran ejemplos de código Python que utiliza Pandas. El código se ejecuta en

un nuevo cuaderno de Jupyter.

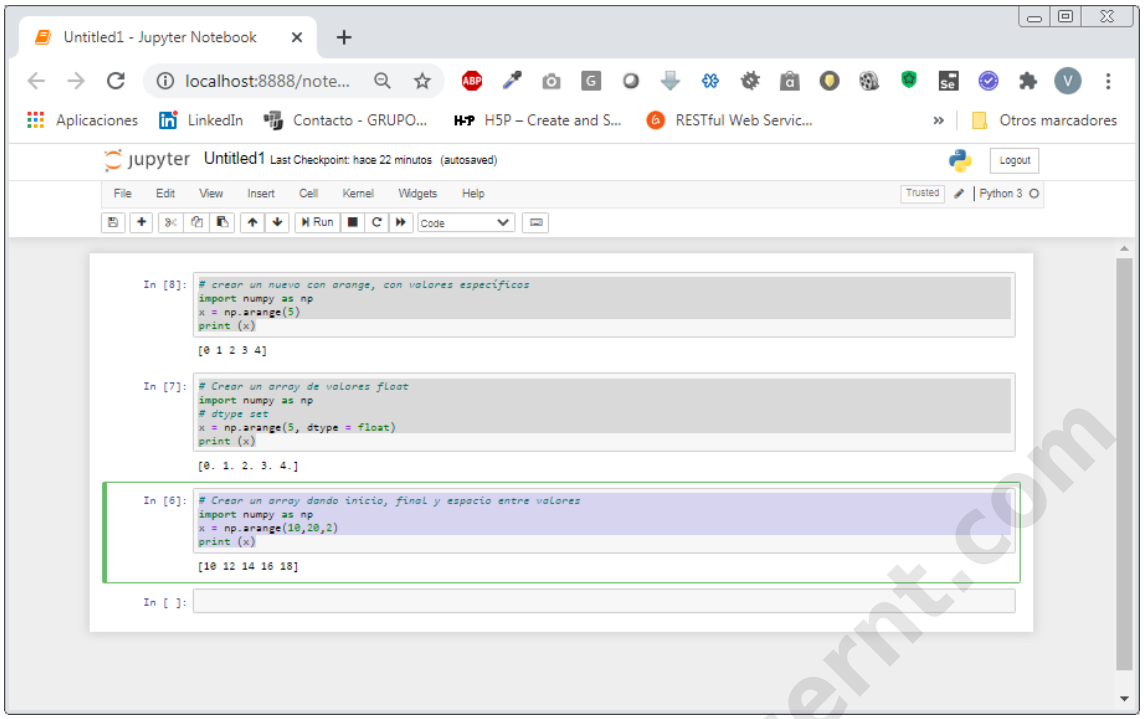
```
# crear un nuevo ndarray

import numpy as np

a = np.array([1,2,3])

print (a)

# más de una dimensión

import numpy as np

a = np.array([[1, 2], [3, 4]])

print (a)

# parámetro dtype para forzar un tipo de datos complejo en el array

import numpy as np

a = np.array([1, 2, 3], dtype = complex)

print (a)
```




formaciononline.infosernt.com

```
# crear un nuevo array con arange, con valores específicos

import numpy as np

x = np.arange(5)

print (x)

# Crear un array de valores float

import numpy as np

# dtype set

x = np.arange(5, dtype = float)

print (x)

# Crear un array dando inicio, final y espacio entre valores

import numpy as np

x = np.arange(10,20,2)

print (x)
```

formaciononline.infosernt.com




# Mostrar la forma del array

import numpy as np

a = np.array([[1,2,3],[4,5,6]])

print (a.shape)

# Redimensionar el array

import numpy as np

a = np.array([[1,2,3],[4,5,6]])

a.shape = (3,2)

print (a)

# Remodelar el array

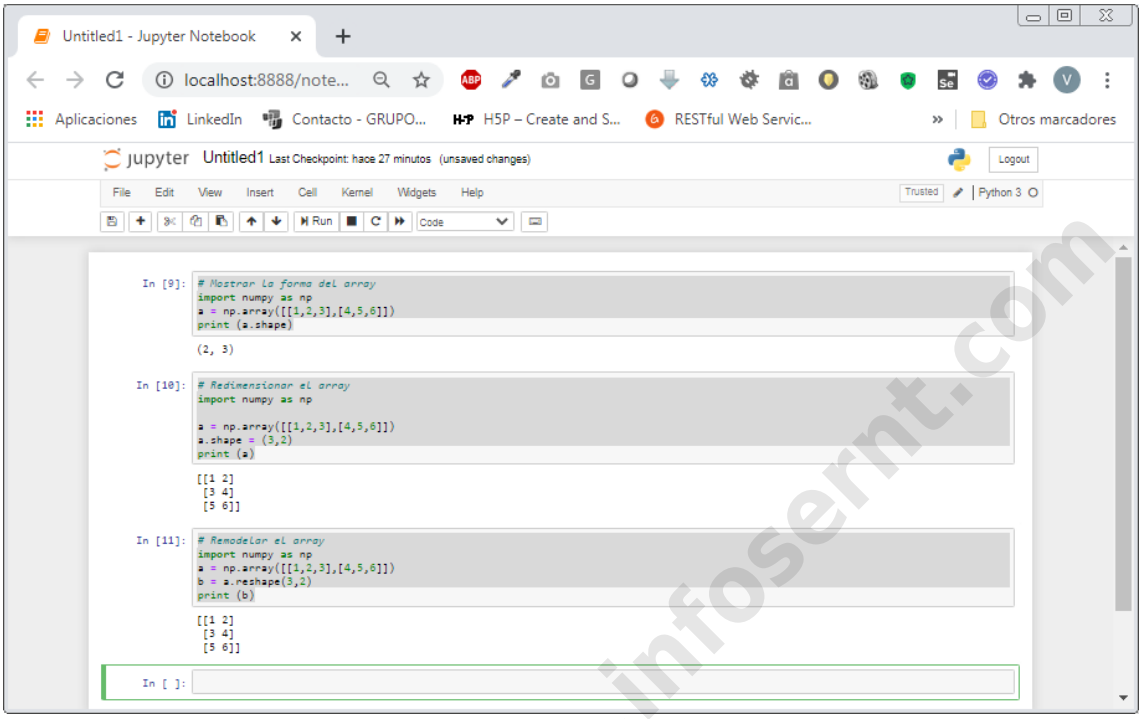
import numpy as np

a = np.array([[1,2,3],[4,5,6]])

formaciononline.infosernt.com

b = a.reshape(3,2)

print (b)




# Iterar sobre un array

import numpy as np

a = np.arange(0,60,5)

a = a.reshape(3,4)

print ('Array original:')

print (a)

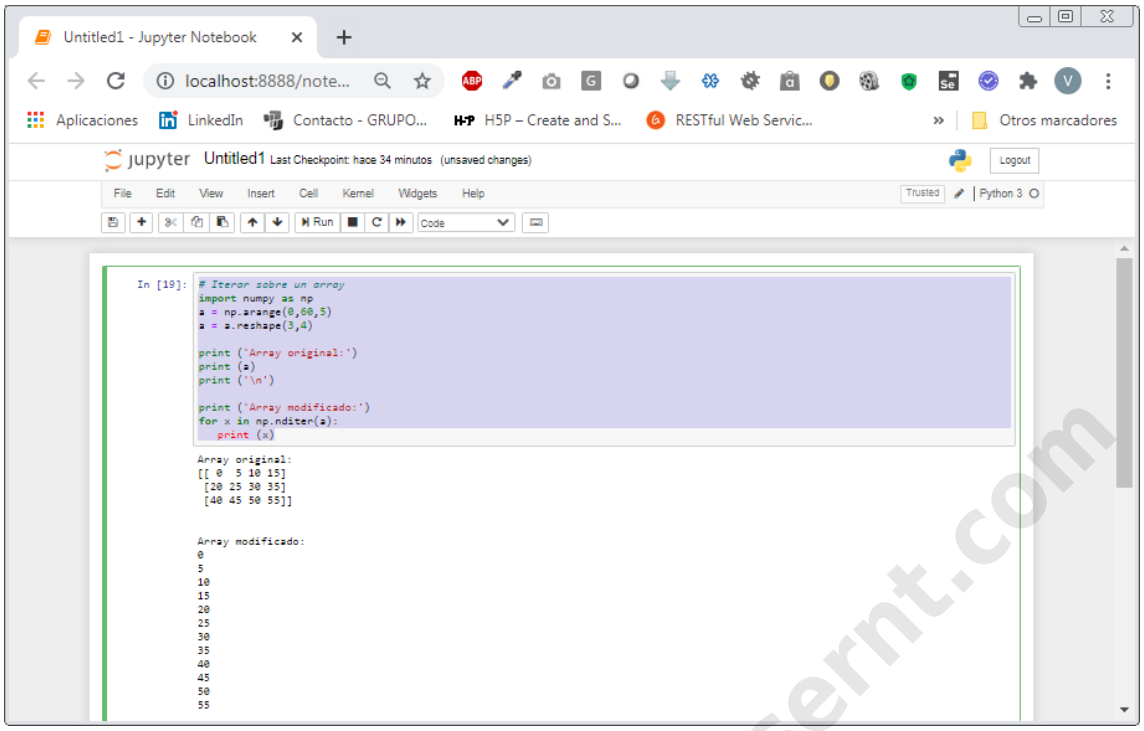
print ('\n')

print ('Array modificado:')

for x in np.nditer(a):

print (x)

formaciononline.infosernt.com




# Modificar el orden de las iteraciones

import numpy as np

a = np.arange(0,60,5)

a = a.reshape(3,4)

print ('Array original:')

print (a)

print ('\n')

print ('Transposición del array:')

b = a.T

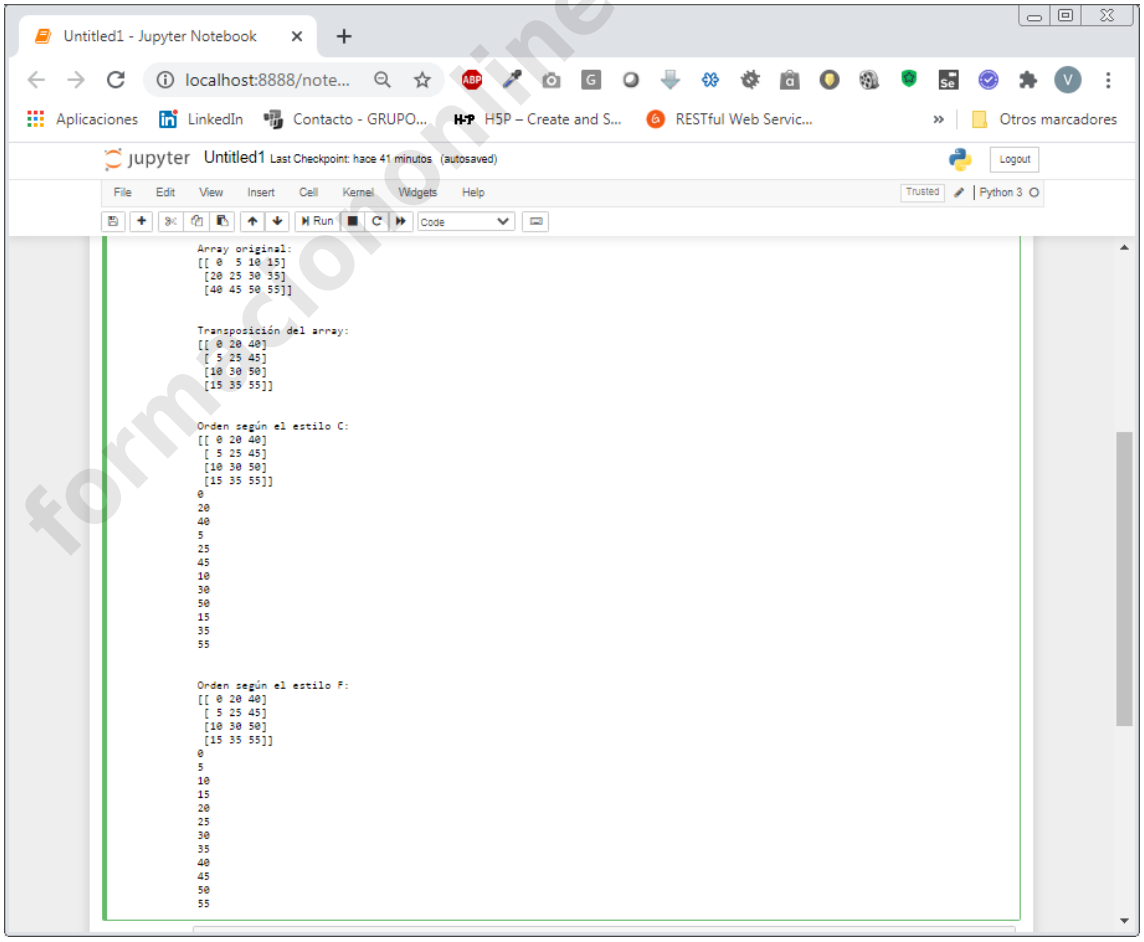
print (b)

print ('\n')

print ('Orden según el estilo C:')

formaciononline.infosernt.com

```
c = b.copy(order = 'C')

print (c)

for x in np.nditer(c):

print (x,)

print ('\n')

print ('Orden según el estilo F:')

c = b.copy(order = 'F')

print (c)

for x in np.nditer(c):

print (x,)
```




formaciononline.infosernt.com

**Pandas**

Pandas es una librería que aporta a **Python** unas estructuras de datos fáciles de usar y muy flexibles, que permiten trabajar con la información de forma muy eficiente. Además, implementa un gran número de funciones esenciales para el análisis de **datos**.

Como cualquier otra librería de Python, cuenta con una página oficial desde donde se puede descargar las últimas versiones, obtener documentación o participar en su comunidad.

En concreto, esta librería relacionada con la analítica de datos, funciona especialmente bien cuando hay que trabajar con datos heterogéneos, series temporales o con grandes volúmenes de datos, pues se obtiene un buen rendimiento.

Estructuras de datos de utilidad implementadas en Pandas.

Como se comentaba al comienzo, Pandas dispone de unas estructuras de datos muy flexibles con las que trabajar. Vamos a comentar dos de ellas que serán de utilidad para el análisis de datos.

**Series**

Estamos ante la estructura de datos más simple que ofrece Pandas, se trata de arrays unidimensionales con indexación, similar a los diccionarios, de hecho, pueden generarse a partir de diccionarios o listas.

Por tanto, las series nos permiten almacenar arrays o vectores con índice o etiqueta. Si no se asigna una etiqueta o índice, se numerará con un índice de forma interna.

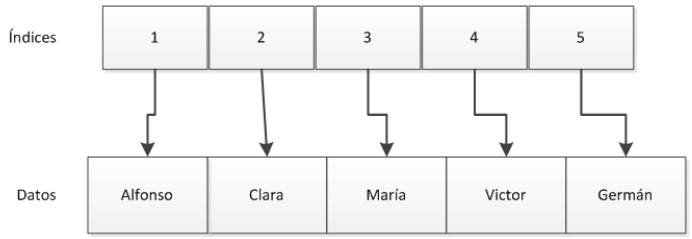
Un objeto definido como una Serie, tiene dos componentes:

- Índice.
- Vector de datos.

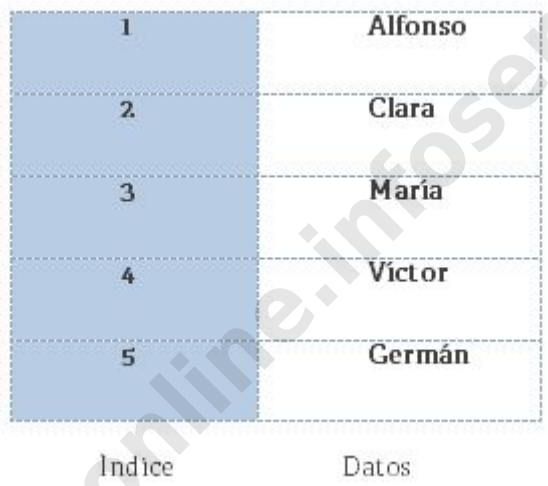
Internamente, ambos componentes son listas con la misma longitud, de manera que el primer elemento de la lista de índices se corresponde con el primer elemento de lista de datos y así de forma sucesiva. La lista de índices debe tener por lo general, valores únicos y ordenados pues se usan para acceder a los valores de los datos del vector.

formaciononline.infosernt.com

De forma visual, la estructura del tipo de **dato Series** sería de la forma:




O lo que es lo mismo:



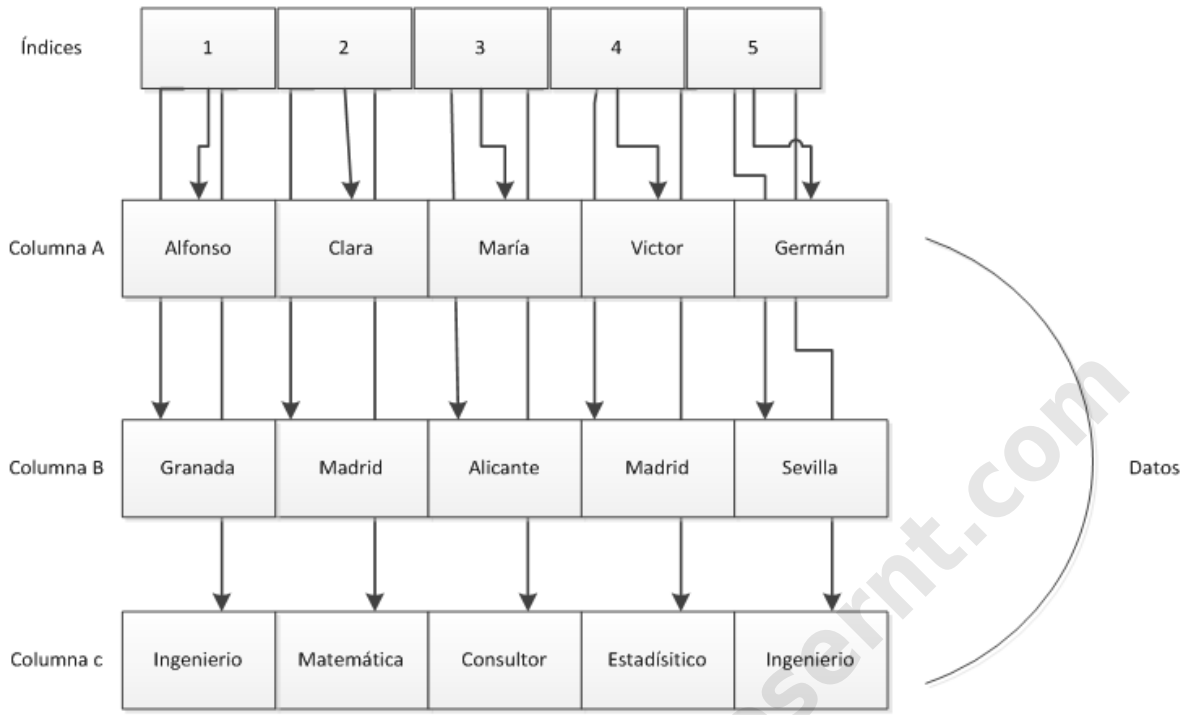

**Dataframes**

Son estructuras de datos que crean objetos similares a una tabla de bases de **datos relacional**. Un objeto de la clase DataFrame se construye a partir de un objeto de tipo Series, ya que en cada columna de la tabla tenemos un objeto Series que comparten un único índice (lo que sería la clave principal en una base de datos).

Para crear un objeto DataFrame, se puede hacer a partir de listas, diccionarios, diccionarios de listas o de Series, y podemos especificar tanto el nombre de las filas (index) como de las columnas (columns).

De forma visual, la estructura del tipo de dato DataFrame sería de la forma:

formaciononline.infosernt.com




O lo que es lo mismo:

| | A | B | C |
|---|---|---|---|
| 1 | Alfonso | Granada | Ingeniero |
| 2 | Clara | Madrid | Matemática |
| 3 | María | Alicante | Consultor |
| 4 | Víctor | Madrid | Estadístico |
| 5 | Germán | Sevilla | Ingeniero |

**Trabajando con Pandas**

De nuevo, antes de comenzar a trabajar con las funcionalidades que ofrece esta librería de Python, debemos comprobar si la tenemos ya instalada en nuestra máquina. Si utilizamos una versión reciente de Anaconda, de nuevo la librería estará instalada por defecto. Podemos comprobarlo siguiendo los mismos pasos que para la librería Numpy tanto para ver si está instalada como para llevar a cabo su instalación en caso contrario.

Igual que por convenio (de la comunidad de desarrolladores) para la librería Numpy siempre escribimos el alias **np**, para Pandas se pone **pd** como alias.

formaciononline.infosernt.com

**Extracción y filtrado de datos**

Para seleccionar y extraer aquellos datos que nos interesan en un determinado momento, y que están contenidos en alguna de las estructuras de datos implementadas en Pandas (Series o DataFrame), disponemos de varios métodos.

**loc:** Se utiliza para seleccionar datos a partir de las etiquetas de las filas y las columnas.

**iloc:** Se utiliza para seleccionar datos a partir de la posición de los mismos dentro de la estructura de datos.

Cuando trabajemos con Series, los métodos anteriores nos devolverán un único valor, y en el caso de trabajar con DataFrames puede devolverse tanto una Serie (si sólo indicamos la posición de la fila), o un valor único (si se indican una fila y una columna).

formaciononline.infosernt.com

```
IPython: C:paula.rochina/Documents

In [9]: from pandas import *

In [10]: s = Series ([1,2,3], index=['a','a','c'])

In [11]: d = DataFrame({'uno' : Series([1,2,3], index=['a','b','c']), 'dos' : S
    ...: eries([1,2,3,4], index=['a','b','c','d'])})

In [12]: s
Out[12]:
a    1
a    2
c    3
dtype: int64

In [13]: d
Out[13]:
   dos  uno
a    1  1.0
b    2  2.0
c    3  3.0
d    4  NaN

In [14]: s.loc['a']
Out[14]:
a    1
a    2
dtype: int64

In [15]: d.loc['a']
Out[15]:
dos    1.0
uno    1.0
Name: a, dtype: float64

In [16]: d.loc['a', 'dos']
Out[16]: 1.0

In [17]: s.iloc[2]
Out[17]: 3

In [18]: d.iloc[1]
Out[18]:
dos    2.0
uno    2.0
Name: b, dtype: float64

In [19]: d.iloc[1,0]
Out[19]: 2.0

In [20]: s.ix[1]
Out[20]: 2

In [21]: d.ix[1]
Out[21]:
dos    2.0
uno    2.0
Name: b, dtype: float64

In [22]: d.ix['b',1]
Out[22]: 2.0

In [23]: _
```

Como hemos visto, con los métodos anteriores podemos seleccionar aquellos datos que nos interesan gracias a la indexación, pero puede también resultar de utilidad el filtrado de los datos, es decir seleccionar los elementos que cumplan una determinada condición. Para esto, podemos hacer uso de los **filtros booleanos**, que permiten la comparación con un valor de referencia. Veamos algunos ejemplos.

```
IPython: C:paula.rochina/Documents

In [23]: filtro_serie = s[s>=3]

In [24]: filtro_serie
Out[24]:
c    3
dtype: int64

In [25]: filtro_dataframe = d[d['dos']>1]

In [26]: filtro_dataframe
Out[26]:
   dos  uno
b    2  2.0
c    3  3.0
d    4  NaN

In [27]:
```

Visualización de los datos.

Para ver y explorar los datos con Pandas, podemos usar entre otros los siguientes métodos:

**describe:** Imprime una tabla resumen de datos estadísticos sobre los valores, como mínimo, máximo, media, etc., por columna.

**head y tail:** Muestran las primeras y últimas 'x' filas respectivamente de una tabla, en función del valor que le pasemos como argumento.

**sort_values():** Ordena los valores a lo largo de cualquiera de los ejes.

**columns**: Devuelve una lista de las columnas del DataFrame.

Se deja como ejercicio que el alumno pruebe los métodos descritos mediante un ejemplo concreto de DataFrame.

**Datos NAN**

Como hemos comprado, en alguno de los ejemplos anteriores, se nos muestra el tipo de dato NAN para algunos valores de los objetos creados. Pandas, hace uso de numpy.nan para representar datos faltantes, es decir, aquellos valores nulos o que aún no se han definido. Se deben tener en consideración estos valores, ya que algunas operaciones tratan de una forma especial este valor, así por ejemplo las operaciones aritméticas consideran que NAN = 0, mientras que **count** (que nos devuelve el número de filas para una columna) no la considera.

formaciononline.infosernt.com

Existen definidas algunas funciones para trabajar con este tipo de datos, que nos permiten entre cosas, modificar su valor u omitirlos.

**dropna:** Omite todas las filas que tienen nan.

**isnull:** Devuelve una tabla con valores true o false en función si el valor es nan o no para un determinado campo.

**fillna(val):** Cambia los valores de nan por el contenido de val.

**Lectura/escritura de archivos**

Pandas dispone de un conjunto muy completo de funciones de lectura y escritura de datos desde archivos. Soporta formatos como CVS, XLS y bases de datos como MySQL.

Para esto Pandas implementa la función **read** (para lectura de archivos) y **to** (para escritura).

Ejemplos de código con Pandas.

```
# Crear una serie a partir de un ndarray

import pandas as pd

import numpy as np

data = np.array(['a','b','c','d'])

s = pd.Series(data)

print (s)

# Añadir índices a la serie a partir de un ndarray

import pandas as pd

import numpy as np

data = np.array(['a','b','c','d'])

s = pd.Series(data,index=[100,101,102,103])
```

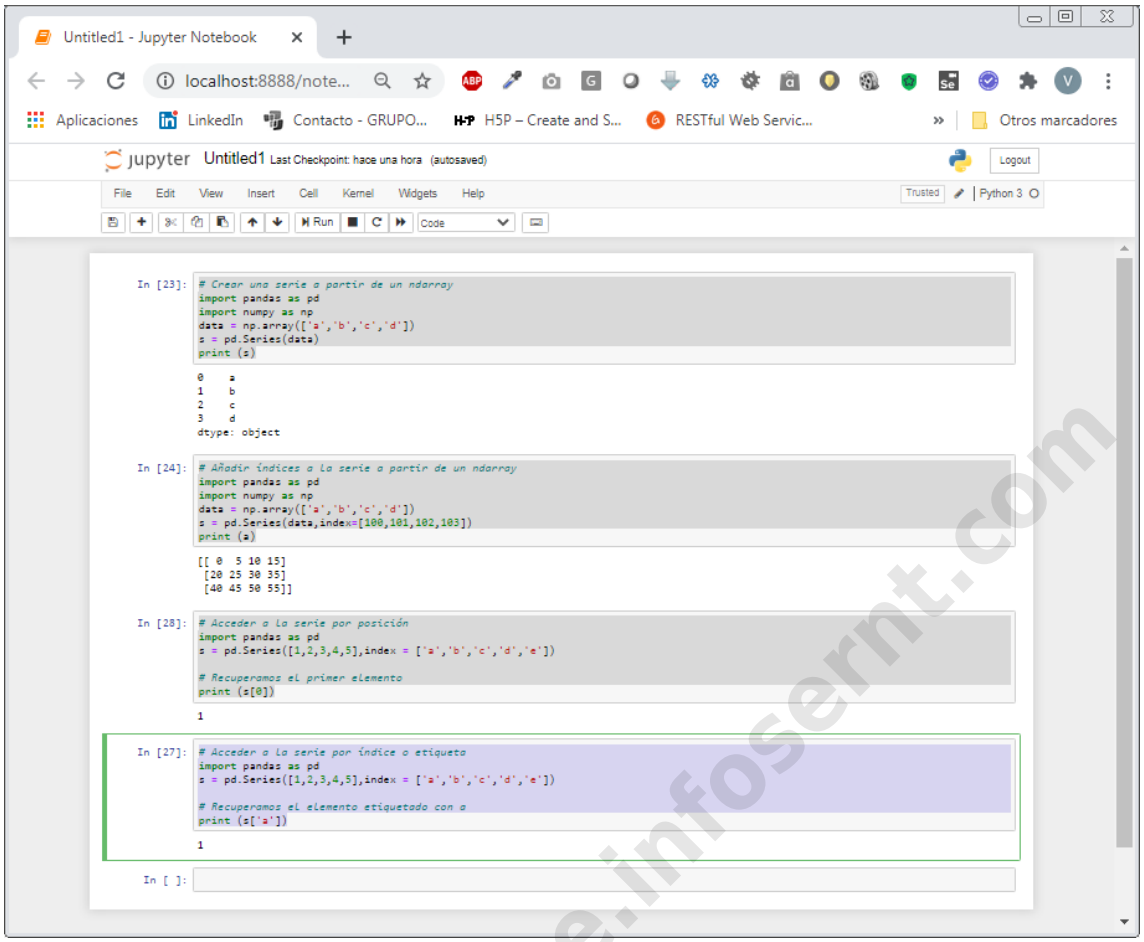
formaciononline.infosernt.com

```
print (s)

# Acceder a la serie por posición

import pandas as pd

s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

# Recuperamos el primer elemento

print (s[0])

# Acceder a la serie por índice o etiqueta

import pandas as pd

s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

# Recuperamos el elemento etiquetado con a

print (s['a'])
```

Nota: También es posible recuperar varios elementos a la vez, por ejemplo:

```
print (s[['a','c','d']])
```

formaciononline.infosernt.com




# Crear un DataFrame vacío

import pandas as pd

df = pd.DataFrame()

print (df)

# Crear un DataFrame a partir de una lista

data = [1,2,3,4,5]

df = pd.DataFrame(data)

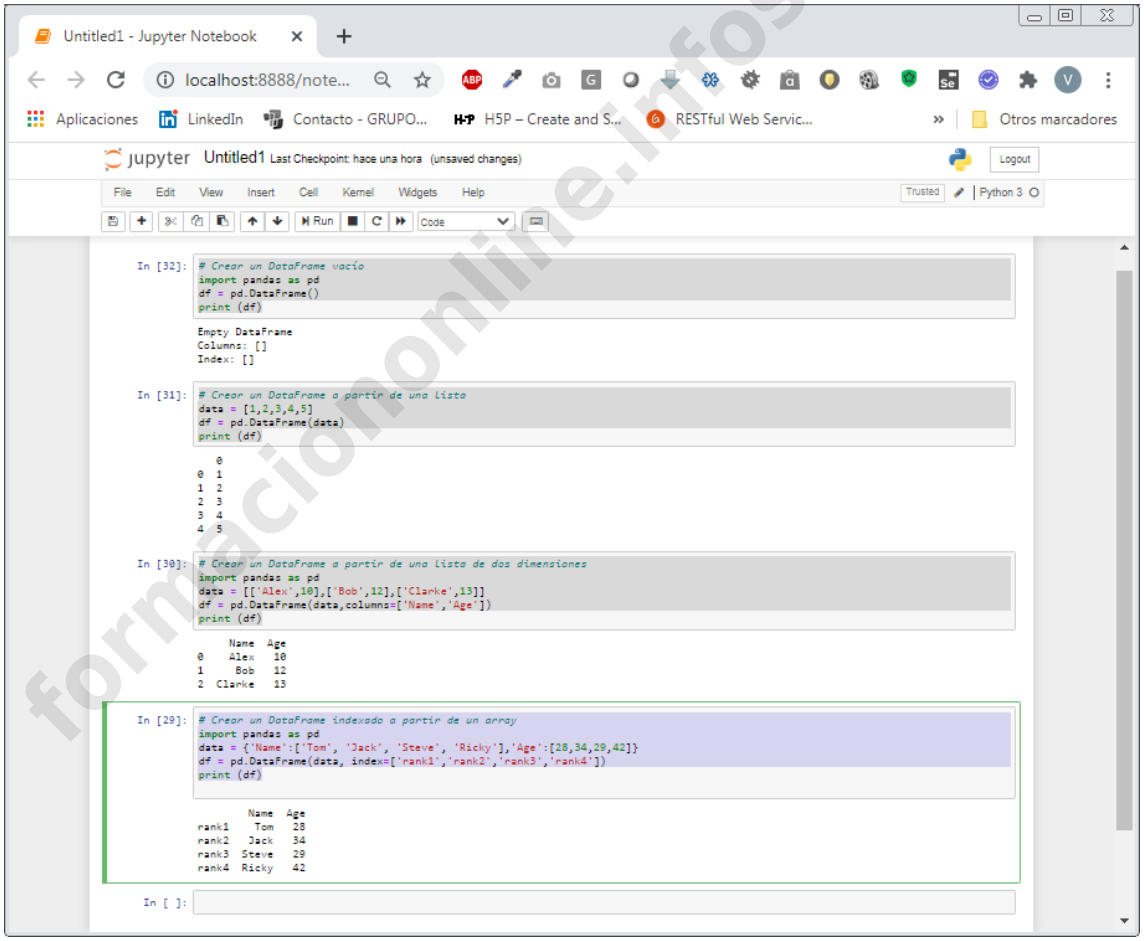
print (df)

# Crear un DataFrame a partir de una lista de dos dimensiones

import pandas as pd

formaciononline.infosernt.com

```
data = [['Alex',10],['Bob',12],['Clarke',13]]

df = pd.DataFrame(data,columns=['Name','Age'])

print (df)

# Crear un DataFrame indexado a partir de un array

import pandas as pd

data = {'Name':['Tom', 'Jack', 'Steve', 'Ricky'],'Age':[28,34,29,42]}

df = pd.DataFrame(data, index=['rank1','rank2','rank3','rank4'])

print (df)
```

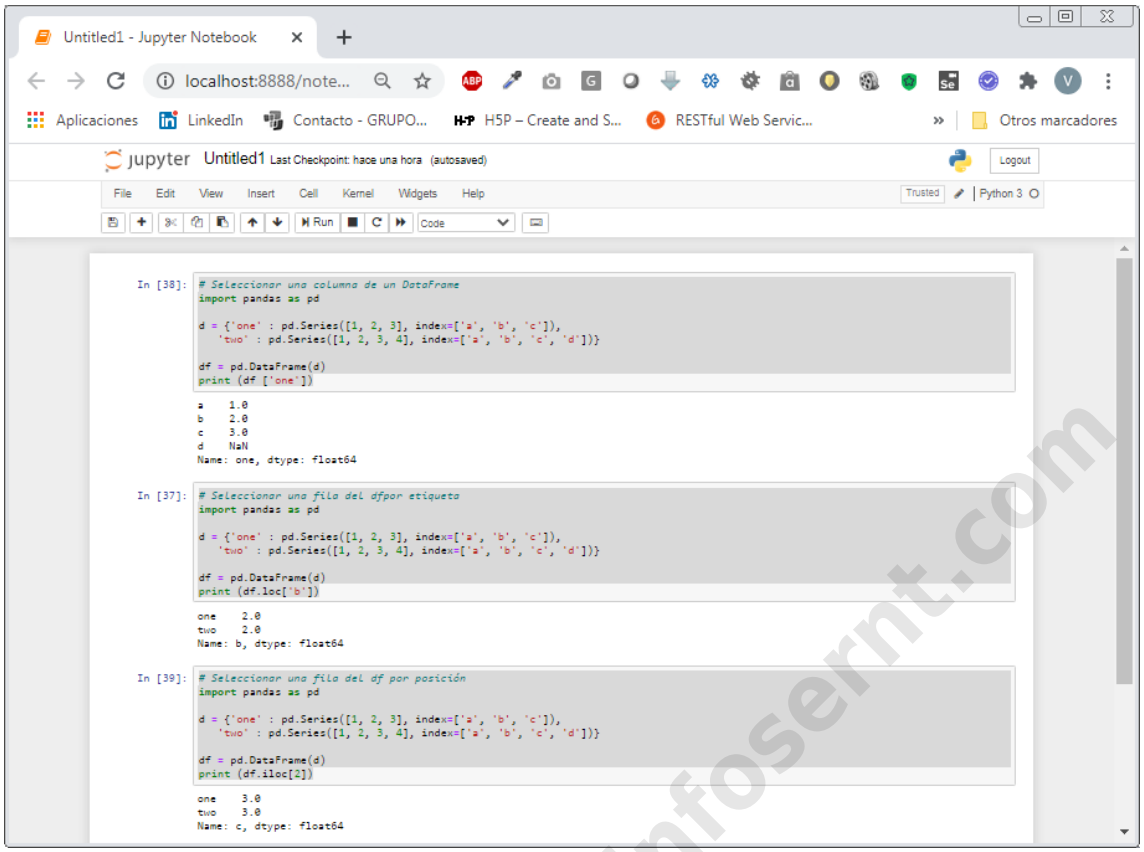



```
# Seleccionar una columna de un DataFrame

import pandas as pd
```

formaciononline.infosernt.com

```
d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),

'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}

df = pd.DataFrame(d)

print (df ['one'])

# Seleccionar una fila del dfpor etiqueta

import pandas as pd

d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),

'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}

df = pd.DataFrame(d)

print (df.loc['b'])

# Seleccionar una fila del df por posición

import pandas as pd

d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),

'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}

df = pd.DataFrame(d)

print (df.iloc[2])
```

formaciononline.infosernt.com




# Borrar columnas de un DataFrame

import pandas as pd

d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),

'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd']),

'three' : pd.Series([10,20,30], index=['a','b','c'])}

df = pd.DataFrame(d)

print ("DataFrame original:")

print (df)

# using del function

print ("Borrar la primera columna utilizando la función DEL:")

del df['one']

formaciononline.infosernt.com

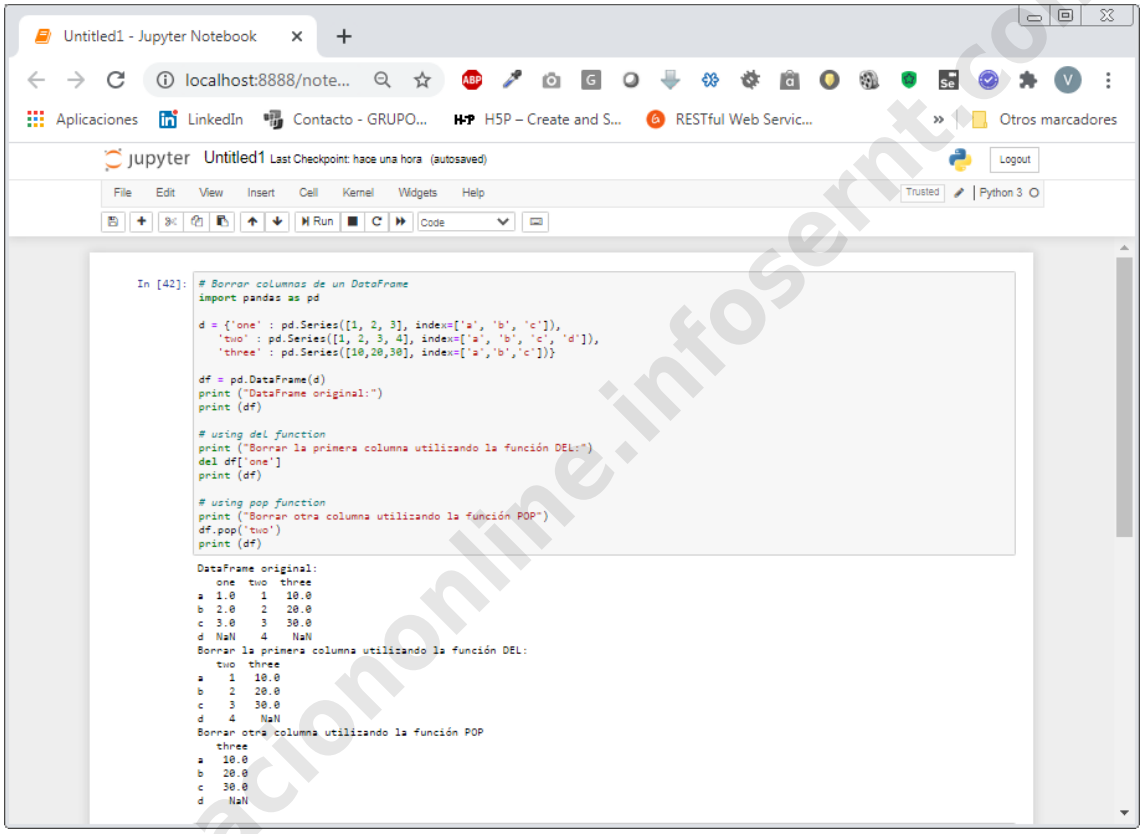
print (df)

# using pop function

print ("Borrar otra columna utilizando la función POP")

df.pop('two')

print (df)




# Seleccionar varias filas de un df

import pandas as pd

d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),

'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}

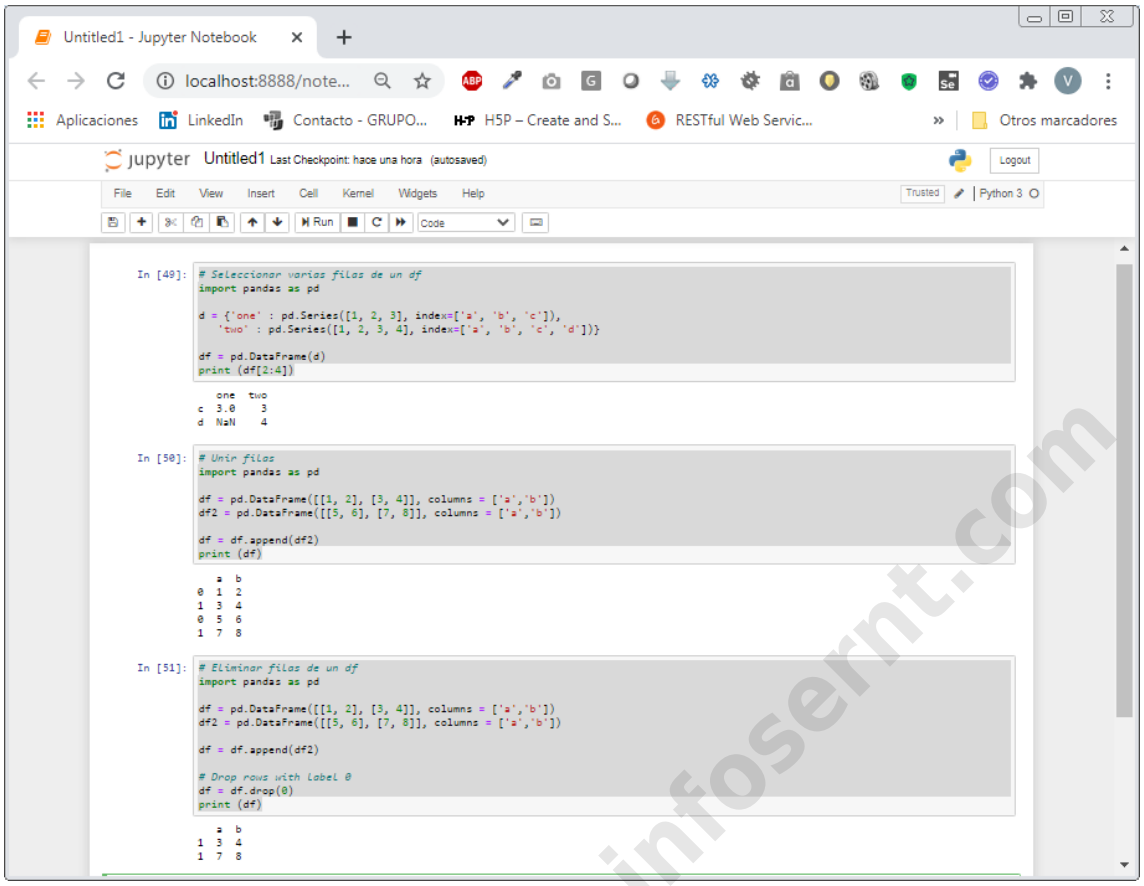
df = pd.DataFrame(d)

print (df[2:4])

formaciononline.infosernt.com

```
# Unir filas

import pandas as pd

df = pd.DataFrame([[1, 2], [3, 4]], columns = ['a','b'])

df2 = pd.DataFrame([[5, 6], [7, 8]], columns = ['a','b'])

df = df.append(df2)

print (df)

# Eliminar filas de un df

import pandas as pd

df = pd.DataFrame([[1, 2], [3, 4]], columns = ['a','b'])

df2 = pd.DataFrame([[5, 6], [7, 8]], columns = ['a','b'])

df = df.append(df2)

# Drop rows with label 0

df = df.drop(0)

print (df)
```

formaciononline.infosernt.com




## Algoritmos de aprendizaje automático

El **aprendizaje automático** (Machine Learning) es una rama de la inteligencia artificial que desarrolla técnicas mediante la implementación de una serie de algoritmos, que permiten aprender a los ordenadores. Es decir, mediante algoritmos matemáticos, se generalizan comportamientos con el fin de encontrar y reconocer patrones a partir de una información facilitada como ejemplo.

Estos algoritmos de aprendizaje automático (que van desde redes bayesanas, redes neuronales artificiales, árboles de decisión, etc.), nos van a permitir por tanto ser capaces de “predecir” el comportamiento futuro de nuestros datos a partir de lo que ha ocurrido en el pasado y de ahí su importancia como herramienta de minería de datos.

### Scikit-learn

Podemos decir que es una de las principales librerías que existen para trabajar con Machine Learning, entre sus ventajas podemos resaltar que se pueden extraer datos de varios repositorios y además cuenta con ciertas bases de datos de muestra que ayudan con la práctica en el uso de los

formaciononline.infosernt.com

algoritmos de aprendizaje automático.

Estamos ante una librería que se centra en el modelado de los datos y no en su manipulación, ya que para esto contamos con Numpy y Pandas. Por otro lado, tiene implementados tanto algoritmo de aprendizaje supervisado como no supervisado, por lo que podemos utilizarla para clasificaciones, extracción de características, regresiones, agrupaciones, reducción de dimensiones, selección de modelos o preprocesamiento.

Para comprobar si disponemos de esta librería en nuestro entorno de programación e instalarla en caso de que no sea así, seguiremos los mismos pasos descritos líneas más arriba para la instalación de paquetes en Python.

**Métodos implementados en scikit-learn**

Como cualquier otra librería de Python, tiene una gran comunidad detrás que se encarga de implementar nuevas funcionalidades, así como actualizar las que ya tiene, por lo que en el momento de trabajar con scikit-learn lo mejor es revisar antes su sitio oficial y comprobar qué algoritmos tiene implementados.

Actualmente podemos dividir los algoritmos implementados, dentro de 6 módulos:

- Clasificación.
- Regresión.
- Clustering.
- Selección automática de modelos.
- Preprocesamiento.
- Reducción de la dimensionalidad.

**Ejemplos de código con scikit-learn**

```
# Cargar la base de datos Iris

from sklearn.datasets import load_iris

iris = load_iris()
```

formaciononline.infosernt.com

X = iris.data

y = iris.target

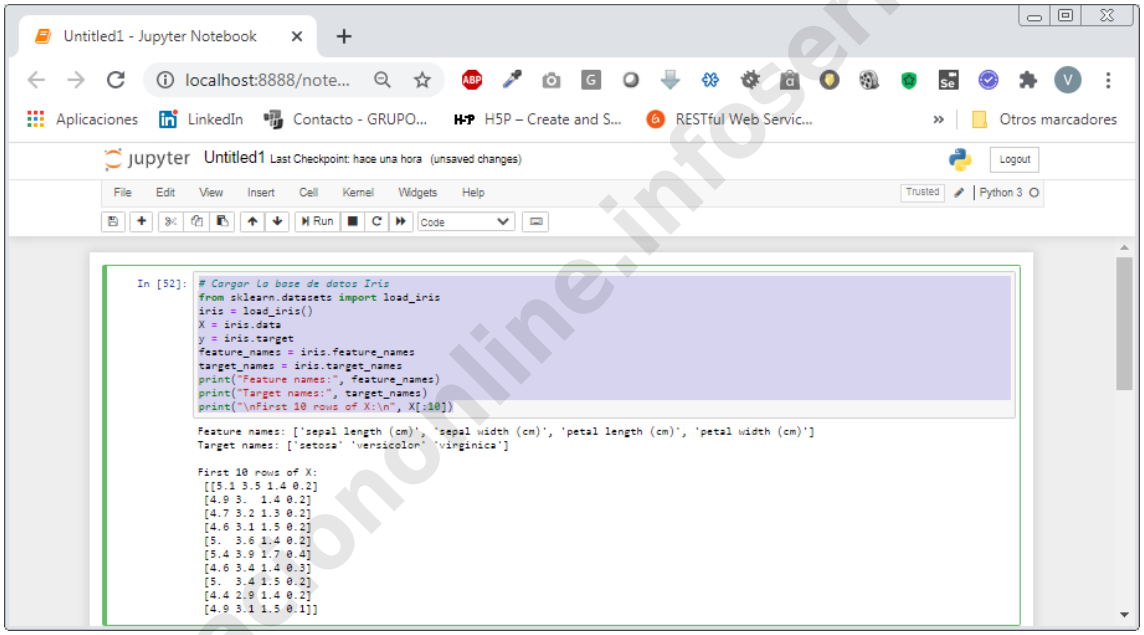
feature_names = iris.feature_names

target_names = iris.target_names

print("Feature names:", feature_names)

print("Target names:", target_names)

print("\nFirst 10 rows of X:\n", X[:10])




# Dividir el conjunto de datos en un conjunto de entrenamiento para entrenar el modelo

# y un conjunto de pruebas (test) para probar el modelo

#Después de puede evaluar la exactitud del modelo

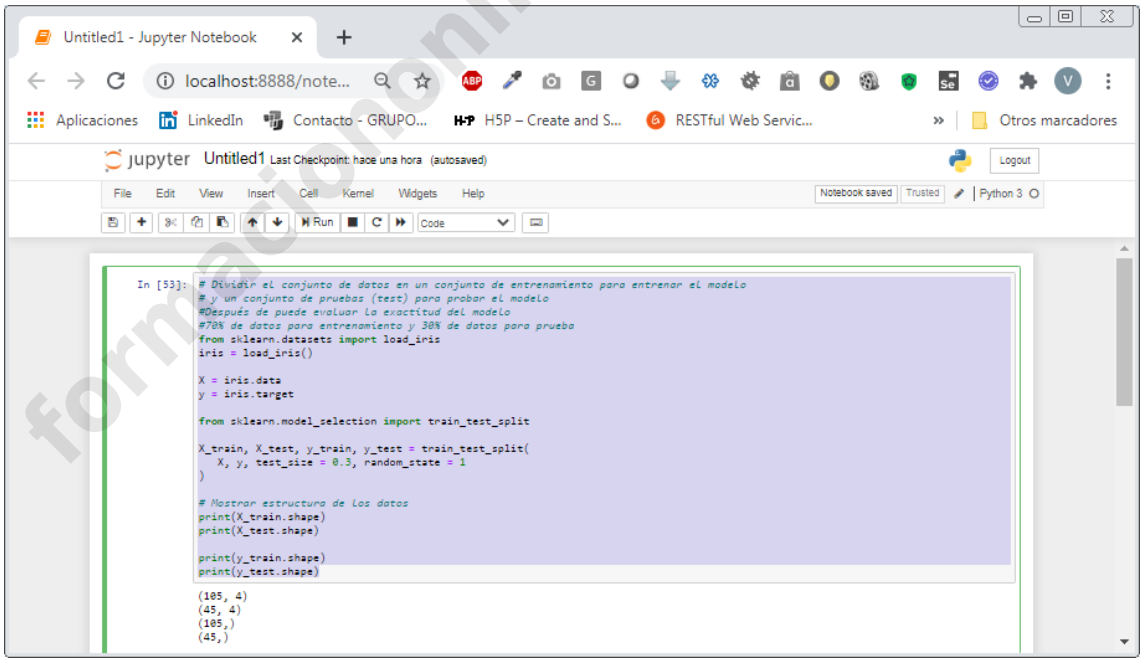
#70% de datos para entrenamiento y 30% de datos para prueba

from sklearn.datasets import load_iris

iris = load_iris()

formaciononline.infosernt.com

```
X = iris.data

y = iris.target

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

X, y, test_size = 0.3, random_state = 1

)

# Mostrar estructura de los datos

print(X_train.shape)

print(X_test.shape)

print(y_train.shape)

print(y_test.shape)
```

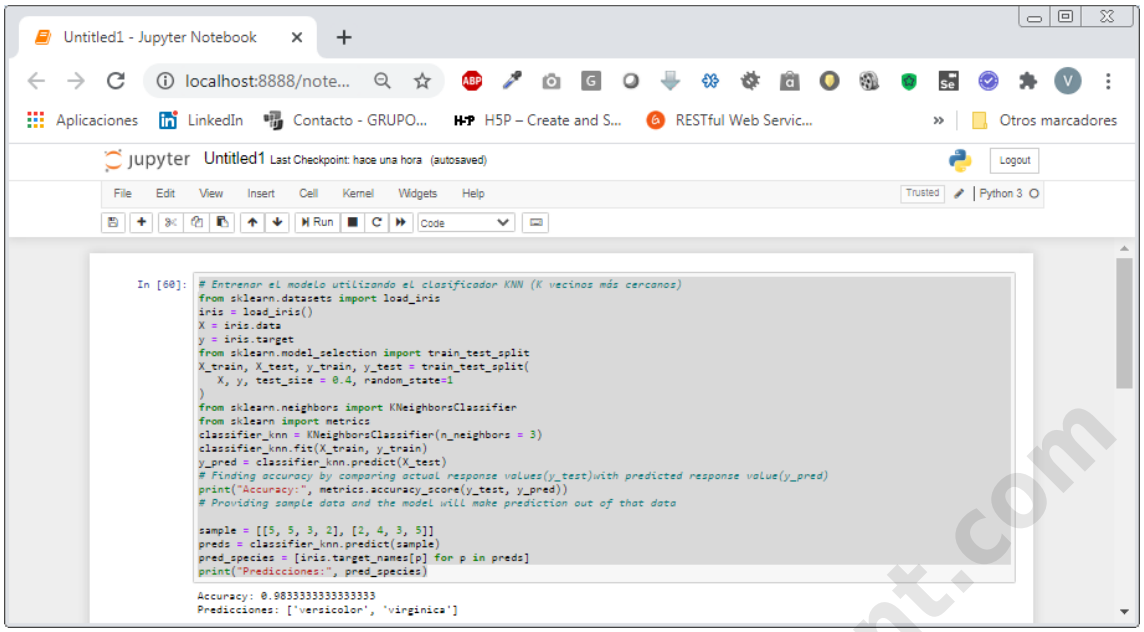



```
# Entrenar el modelo utilizando el clasificador KNN (K vecinos más cercanos)

from sklearn.datasets import load_iris
```

formaciononline.infosernt.com

```
iris = load_iris()

X = iris.data

y = iris.target

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

X, y, test_size = 0.4, random_state=1

)

from sklearn.neighbors import KNeighborsClassifier

from sklearn import metrics

classifier_knn = KNeighborsClassifier(n_neighbors = 3)

classifier_knn.fit(X_train, y_train)

y_pred = classifier_knn.predict(X_test)

# Finding accuracy by comparing actual response values(y_test)with predicted response
value(y_pred)

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

# Providing sample data and the model will make prediction out of that data

sample = [[5, 5, 3, 2], [2, 4, 3, 5]]

preds = classifier_knn.predict(sample)

pred_species = [iris.target_names[p] for p in preds]

print("Predicciones:", pred_species)
```

formaciononline.infosernt.com




## Theano

Theano es una biblioteca de **Python** que proporciona un conjunto de funciones para construir **redes profundas** que entrenan rápidamente en nuestra máquina.

Se desarrolló en la Universidad de Montreal, Canadá, bajo el liderazgo de Yoshua Bengio, un pionero de la red profunda.

Theano nos permite definir y evaluar expresiones matemáticas con vectores y matrices que son matrices rectangulares de números.

Técnicamente hablando, tanto las redes neuronales como los datos de entrada pueden representarse como matrices y todas las operaciones netas estándar pueden redefinirse como operaciones matriciales. Esto es importante ya que las computadoras pueden realizar operaciones matriciales muy rápidamente.

Podemos procesar múltiples valores de matriz en paralelo y si construimos una red neuronal con esta estructura subyacente, podemos usar una sola máquina con una GPU para entrenar redes

formaciononline.infosernt.com

enormes en una ventana de tiempo razonable.

Sin embargo, si usamos Theano, tenemos que construir la red profunda desde cero. La biblioteca no proporciona una funcionalidad completa para crear un tipo específico de red profunda.

En cambio, tenemos que codificar todos los aspectos de la red profunda como el modelo, las capas, la activación, el método de entrenamiento y cualquier método especial para detener el sobreajuste.

Sin embargo, la buena noticia es que Theano permite construir nuestra implementación sobre una parte superior de las funciones vectorizadas, proporcionándonos una solución altamente optimizada.

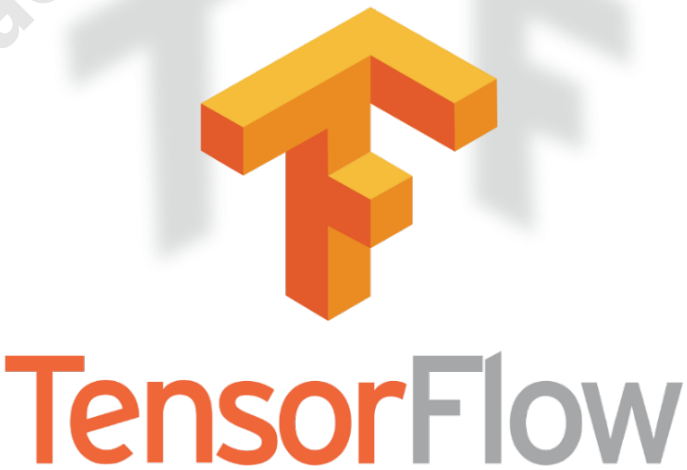
Hay muchas otras bibliotecas que amplían la funcionalidad de Theano. TensorFlow y Keras se pueden usar con Theano como backend.

**TensorFlow**

Google TensorFlow es una biblioteca de Python. Esta biblioteca es una excelente opción para crear aplicaciones de aprendizaje profundo de grado comercial.

TensorFlow surgió de otra biblioteca DistBelief V2 que era parte de Google Brain Project. Esta biblioteca tiene como objetivo extender la portabilidad del aprendizaje automático para que los modelos de investigación puedan aplicarse a aplicaciones de grado comercial.

formaciononline.infosernt.com




Al igual que la biblioteca **Theano**, **TensorFlow** se basa en gráficos computacionales en los que un

nodo representa datos persistentes u operaciones matemáticas y los bordes representan el flujo de datos entre nodos, que es una matriz o tensor multidimensional; de ahí el nombre de TensorFlow.

La salida de una operación o un conjunto de operaciones se alimenta como entrada en la siguiente.

A pesar de que TensorFlow fue diseñado para redes neuronales, funciona bien para otras redes donde el cálculo se puede modelar como un gráfico de flujo de datos.

TensorFlow también utiliza varias características de Theano, como la eliminación común y de subexpresión, la diferenciación automática, las variables compartidas y simbólicas.

Se pueden construir diferentes tipos de redes profundas utilizando TensorFlow como redes convolucionales, Autoencoders, RNTN, RNN, RBM, DBM / MLP, etc.

Sin embargo, no hay soporte para la configuración de hiperparámetros en TensorFlow. Para esta funcionalidad, podemos usar Keras.

Los **tensores** se utilizan como estructuras de datos básicas en el lenguaje TensorFlow. Los tensores representan los bordes de conexión en cualquier diagrama de flujo llamado Gráfico de flujo de datos.

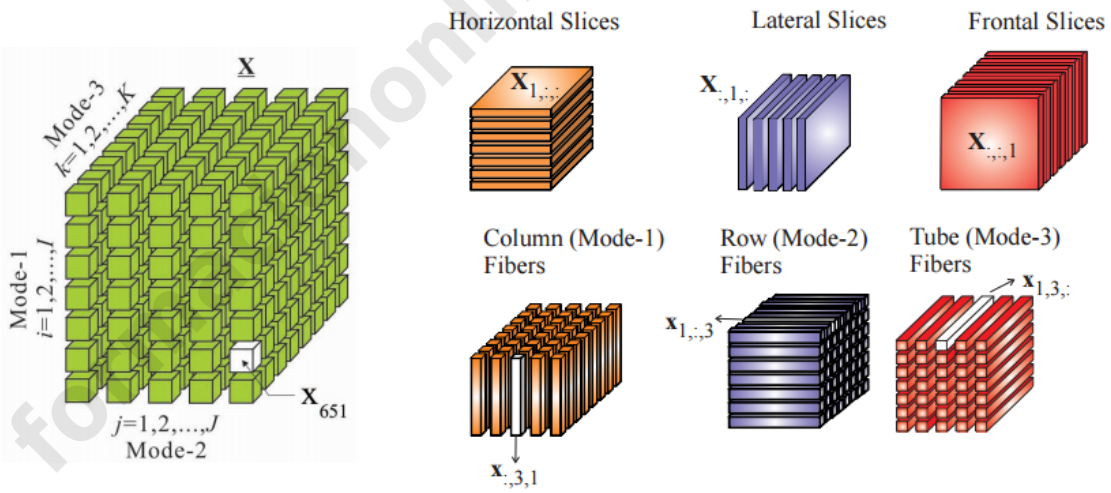



**Figure 2:** A 3rd-order tensor $\underline{\mathbf{X}} \in \mathbb{R}^{I \times J \times K}$, with entries $x_{i,j,k} =$

Los tensores se definen como una matriz o lista multidimensional, y se identifican mediante los siguientes tres parámetros:

- Rango: La unidad de dimensionalidad descrita dentro del tensor se denomina rango. Identifica el número de dimensiones del tensor. El rango de un tensor puede describirse como el orden o

n-dimensiones de un tensor definido.

- Forma: El número de filas y columnas juntas definen la forma de Tensor.
- Tipo: Describe el tipo de datos asignado a los elementos de Tensor.

Consideramos las siguientes actividades para construir un Tensor:

- Construir una matriz n-dimensional.
- Convertir la matriz n-dimensional.

**Keras**

Keras es una biblioteca de Python potente y fácil de usar para desarrollar y evaluar modelos de aprendizaje profundo.

Tiene un diseño minimalista que nos permite construir una red capa por capa; entrenarlo y ejecutarlo.

Envuelve las eficientes bibliotecas de computación numérica **Theano** y **TensorFlow** y nos permite definir y entrenar modelos de redes neuronales en unas pocas líneas cortas de código.

Es una API de red neuronal de alto nivel, que ayuda a hacer un uso amplio del aprendizaje profundo y la inteligencia artificial. Se ejecuta sobre una serie de bibliotecas de nivel inferior, incluidas TensorFlow, Theano, etc. El código Keras es portátil; podemos implementar una red neuronal en Keras usando Theano o TensorFlow como back-end sin ningún cambio en el código.

Keras está diseñado para comprender las técnicas de aprendizaje profundo, como la creación de capas para redes neuronales que mantienen los conceptos de formas y detalles matemáticos. La creación de freamework puede ser de los siguientes tipos:

- API secuencial.

formaciononline.infosernt.com

- API funcional.

Consideramos los siguientes ocho pasos para crear un modelo de aprendizaje profundo en Keras:

- Cargar los datos.
- Preprocesar los datos cargados.
- Definición del modelo.
- Compilar el modelo.
- Se ajusta al modelo especificado.
- Evaluarlo.
- Hacer las predicciones requeridas.
- Guardar el modelo.

## Visualización y representación de datos con Matplotlib

Python dispone de una gran cantidad de módulos para la representación y visualización de los datos, pero será objeto de estudio el módulo Matplotlib (que permite la creación de gráficos en 2D y 3D) pues tiene implementada la librería pyplot muy semejante a Matlab, y que nos permitirá una muy buena visualización de los datos.

Matplotlib, se integra a la perfección con los módulos descritos hasta el momento para el análisis de datos, puesto que está implementada y orientada a la generación de gráficos a partir de datos contenidos en listas o arrays.

Una vez instalado el módulo (de nuevo siguiendo los pasos descritos en puntos anteriores), para poder usar sus funcionalidades debemos importarlo. Como alias utilizaremos en este caso **plt**.

Tipos de gráficos con pyplot.

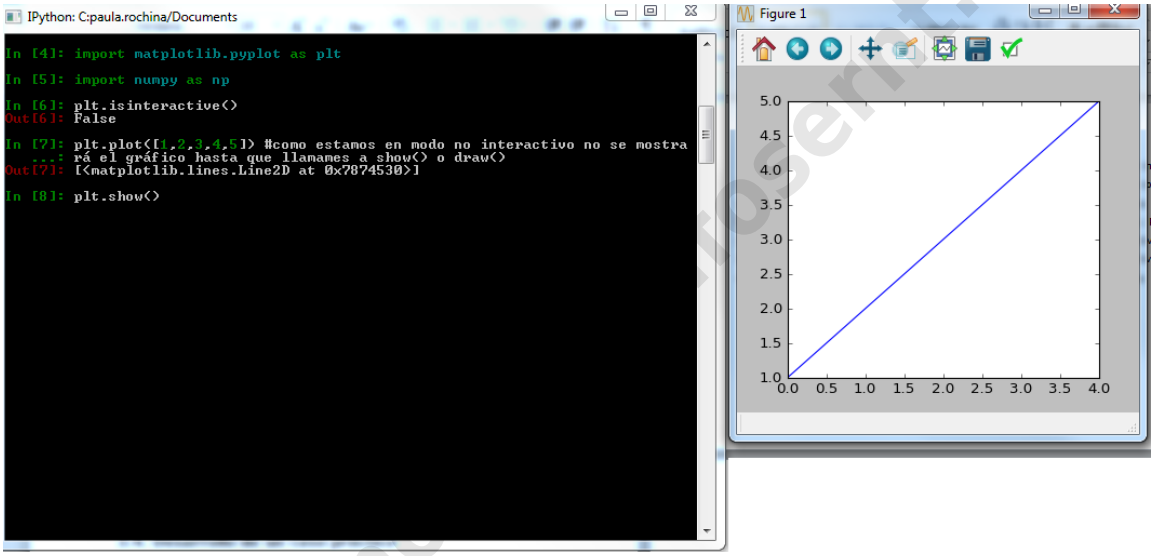
Dentro de **pyplot** disponemos de diferentes **tipos de gráficos**, aunque los más usuales son:

- Gráfico (clásico) de datos 2D: Usamos la función **plot**
- Histogramas: Usamos la función **hist**
- Campos de vectores: Usamos la función **quiver.**

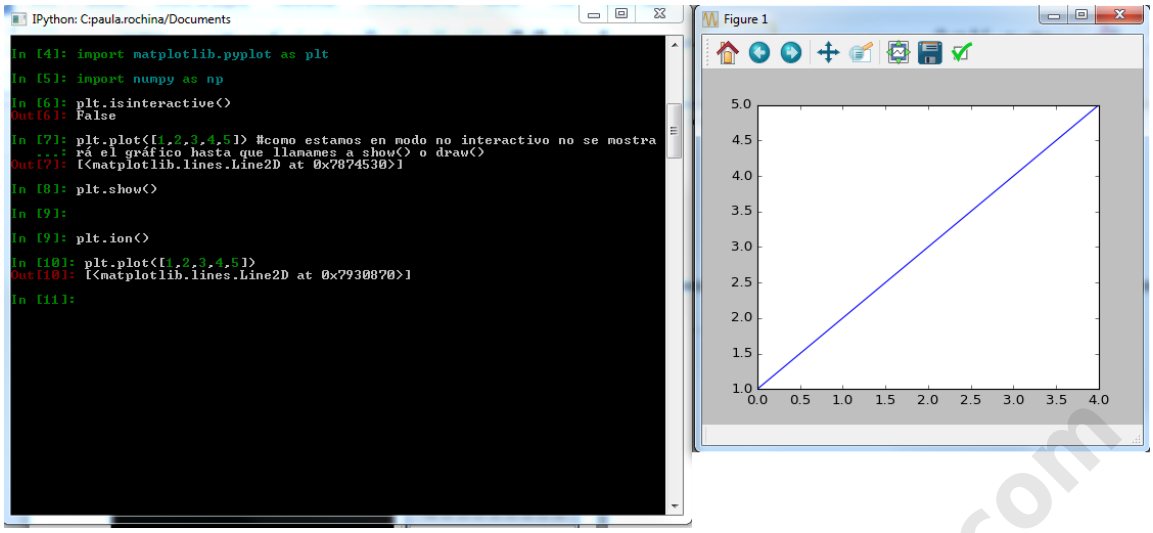
formaciononline.infosernt.com

- Gráficos circulares o de “torta”: Usamos la función **pie**
- Gráficos de barras: Usamos la función **bar**.

Cuando trabajamos con pyplot se puede hacer de dos formas, en modo interactivo, de manera que cada vez que modificamos el código relacionado con el gráfico éste se actualiza, o en modo no interactivo para lo que tendremos que llamar a la función **show()** o **draw()** para mostrar el gráfico modificado. Para conmutar de un modo a otro, podemos usar **ion()** e **ioff()**, que permiten activar y desactivar respectivamente el modo interactivo. Para comprobar si está activo el modo interactivo usamos la función **isinteractive()**, que devuelve *true o false*.




Como se aprecia en la imagen anterior, al tener el modo interactivo desactivado (por defecto), hasta que llamamos a la función show() no se nos muestra el gráfico, el cual nos aparece en una ventana externa que nos permitirá entre cosas guardar el gráfico como imagen para usarlo después en nuestros informes. A continuación, mostramos un ejemplo con el modo interactivo activado.




Como vemos en la imagen anterior, al activar el modo interactivo no es necesario llamar a ninguna función adicional para mostrar el gráfico, además de que la consola nos permite seguir introduciendo código sin necesidad de cerrar la ventana del gráfico, lo que facilita la visualización de los cambios en el gráfico en tiempo real conforme introducimos o modificamos el código.

**Sobreesribiendo gráficos**

Una función muy llamativa de pyplot, es que nos permite dibujar varias gráficas en un mismo gráfico. Por defecto la opción de sobreescribir en mismo gráfico viene activada al iniciar sesión.

A la representación gráfica que se nos abre en una **nueva ventana** cuando trabajamos con las opciones gráficas de **pyplot**, se le llama figura. Por defecto, el nombre será “Figure#’, ya que las figuras están numerados a partir del 1.

Con la función figure, podemos modificar algunas características de la ventana de gráfico, como el nombre, tamaño, resolución, etc. Los argumentos que admite la función son:

| Argumento | Descripción |
|---|---|
| num = None | Numeración de la figura, si num=None las figuras se numeran automáticamente. |
| figsize = (w,h) | W,h tuplas en pulgadas. Define el tamaño de la figura. |
| dpi = x | Establece la resolución de la imagen en puntos por pulgada. |
| facecolor = ‘color’ | Color del fondo para la figura. |
| edegecolor = ‘color’ | Color del borde alrededor del fondo de la figura. |
| frameon = ‘True’ or ‘False | Permite establecer si dibujar la figura en marcos o no. |

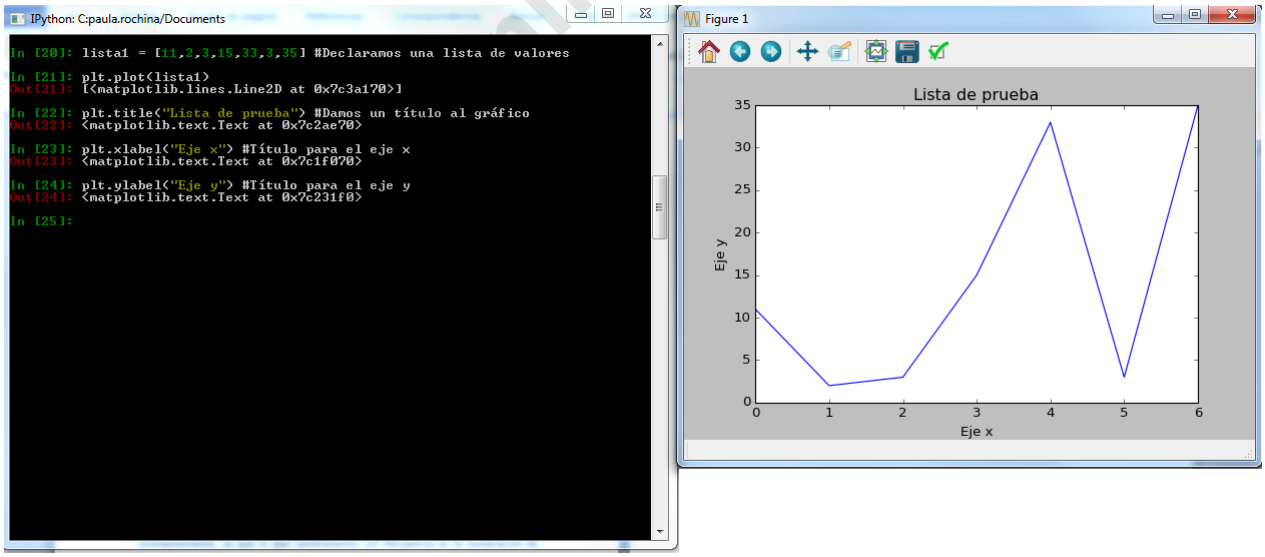
formaciononline.infosernt.com

Todos estos **argumentos** vienen con unos valores por defecto, que podremos especificar en el **archivo de recursos** para no tener que modificarlos constantemente, ya que lo que cambiaremos con frecuencia es la numeración de las figuras, su nombre, etc.

Pyplot, permite también **agregar texto** dentro de una **figura**, para por ejemplo dar nombre a los ejes, un título al gráfico, etc. Para ello se implementan entre otras, las siguientes funciones:

- title("Nombre del gráfico"): Nos permite dar un título al gráfico representado.
- xlabel("Nombre al eje x"): Nos permite dar un título al eje x del gráfico.
- ylabel(Nombre al eje y"): Nos permite dar un título al eje y del gráfico.

Igualmente, podemos incluir la leyenda del gráfico mediante la función **legend().** Para usar esta función, se debe tener en cuenta que previamente hemos debido definir el argumento "label" de la función de dibujo (por ejemplo en plot()) para establecer el literal de la leyenda a mostrar. Así mismo, la función legend(), tiene un argumento (loc) que nos permite posicionar la leyenda en diferentes puntos entorno al gráfico, cuando asignamos algunos de los siguientes valores o una combinación de ellos: upper, lower, centre, left y right.

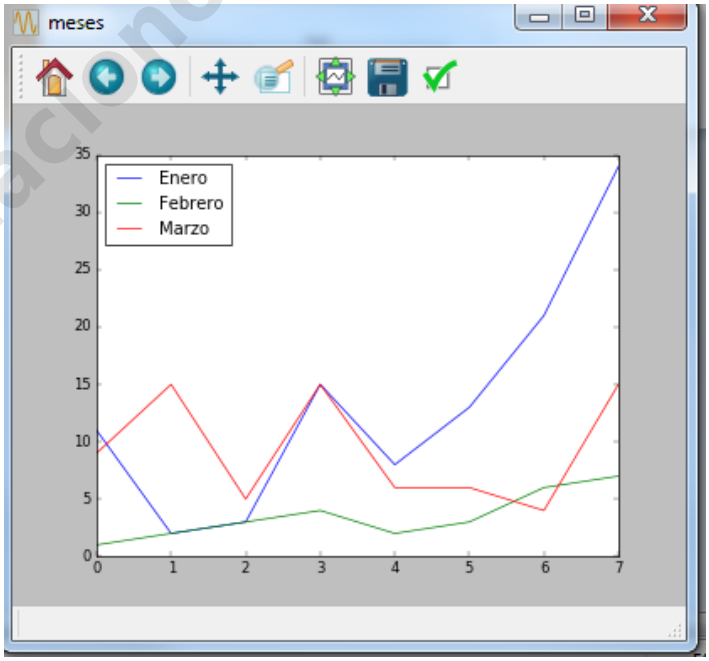



En el ejemplo mostrado en la imagen anterior, hemos dibujado un gráfico al que después hemos dado título y etiquetado sus ejes.

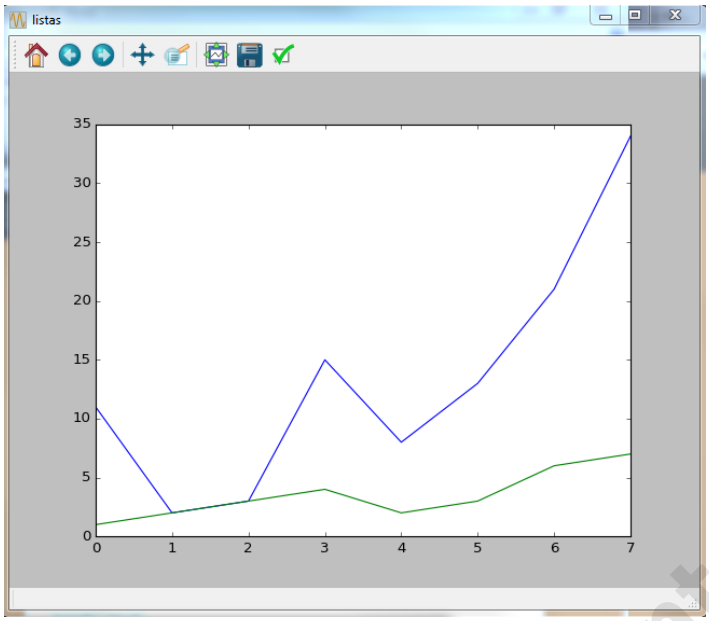
Ahora vamos a crear dos ventanas independientes en las que mostrar gráficos, en cada una de ellas definiremos una serie de características para las gráficas que contenga, e incluiremos la leyenda.

IPython: C:paula.rochina/Documents

```
In [48]: plt.figure('listas', figsize= (8,6), dpi = 80) #creamos la ventana lis
    ...: tas
Out[48]: <matplotlib.figure.Figure at 0x7fec890>

In [49]: plt.figure('meses',dpi = 50) #creamos la ventana meses
Out[49]: <matplotlib.figure.Figure at 0x7b84610>

In [50]: lista1 = [11,2,3,15,8,13,21,34]

In [51]: lista2 = [1,2,3,4,2,3,6,7]

In [52]: lista3 = [9,15,5,15,6,6,4,15]

In [53]: plt.figure('listas') #Indicamos que active la ventana listas que es la
    ...:  que vamos a utilizar
Out[53]: <matplotlib.figure.Figure at 0x7fec890>

In [54]: plt.plot(lista1)
Out[54]: [<matplotlib.lines.Line2D at 0x8740c50>]

In [55]: plt.plot(lista2)
Out[55]: [<matplotlib.lines.Line2D at 0x8746710>]

In [56]: plt.figure('meses')
Out[56]: <matplotlib.figure.Figure at 0x7b84610>

In [57]: plt.plot(lista1, label="Enero")
Out[57]: [<matplotlib.lines.Line2D at 0x8e896b0>]

In [58]: plt.plot(lista2, label="Febrero")
Out[58]: [<matplotlib.lines.Line2D at 0x8ea0090>]

In [59]: plt.plot(lista3, label="Marzo")
Out[59]: [<matplotlib.lines.Line2D at 0x875a9f0>]

In [60]: plt.legend() #incluimos la leyenda
Out[60]: <matplotlib.legend.Legend at 0x8e77090>

In [61]: plt.legend(loc="upper left")
Out[61]: <matplotlib.legend.Legend at 0x8eaf270>

In [62]:
```

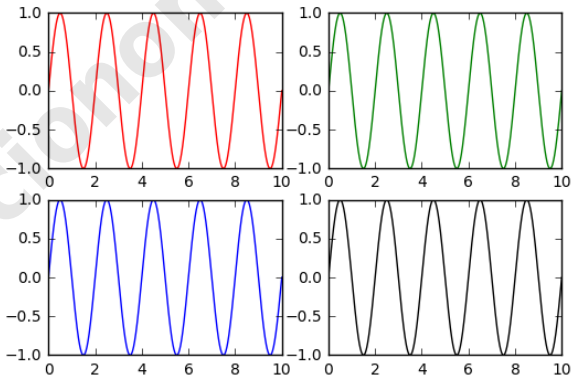



formacionononline




### Subgráficos en una misma figura

Si en lugar de superponer gráficos en una misma figura, queremos crear gráficos independientes pero que se nos muestren en la misma ventana, podemos hacer uso del comando **subplot().** Esta función nos permite dividir el área de dibujo por filas y columnas de manera que luego podemos dibujar en cada espacio definido previamente un gráfico individual. Veamos un ejemplo.




formaciononline.infosernt.com

```python
"""
Código ejemplo de uso de la función "subplot" para mostrar varios gráficos
independientes en una misma figura
"""

import numpy as np                  #Importamos numpy con alias np
import matplotlib.pyplot as plt     #Importamos pyplot con alias plt

#Definimos el período de la gráfica
periodo = 2

#Definimos un array y la función senoidal
x = np.linspace(0,10, 1000)
y = np.sin(2*np.pi*x/periodo)

#creamos la figura
plt.figure('Ejemplo de subplot')

#Dibujamos las diferentes gráficas
plt.subplot (2,2,1)  #indicamos que divida el espacio en 2 filas por 2 columnas y se situe en el primer hueco (1ra fila,1ra columna)
plt.plot(x,y,'r')  #Dibujamos la gráfica de la función senoidal en rojo

plt.subplot (2,2,2)
plt.plot(x,y,'g')

plt.subplot (2,2,3)
plt.plot(x,y,'b')

plt.subplot (2,2,4)
plt.plot(x,y,'k')

#Mostramos en pantalla
plt.show()
```

### Cambiar estilos de líneas de un gráfico

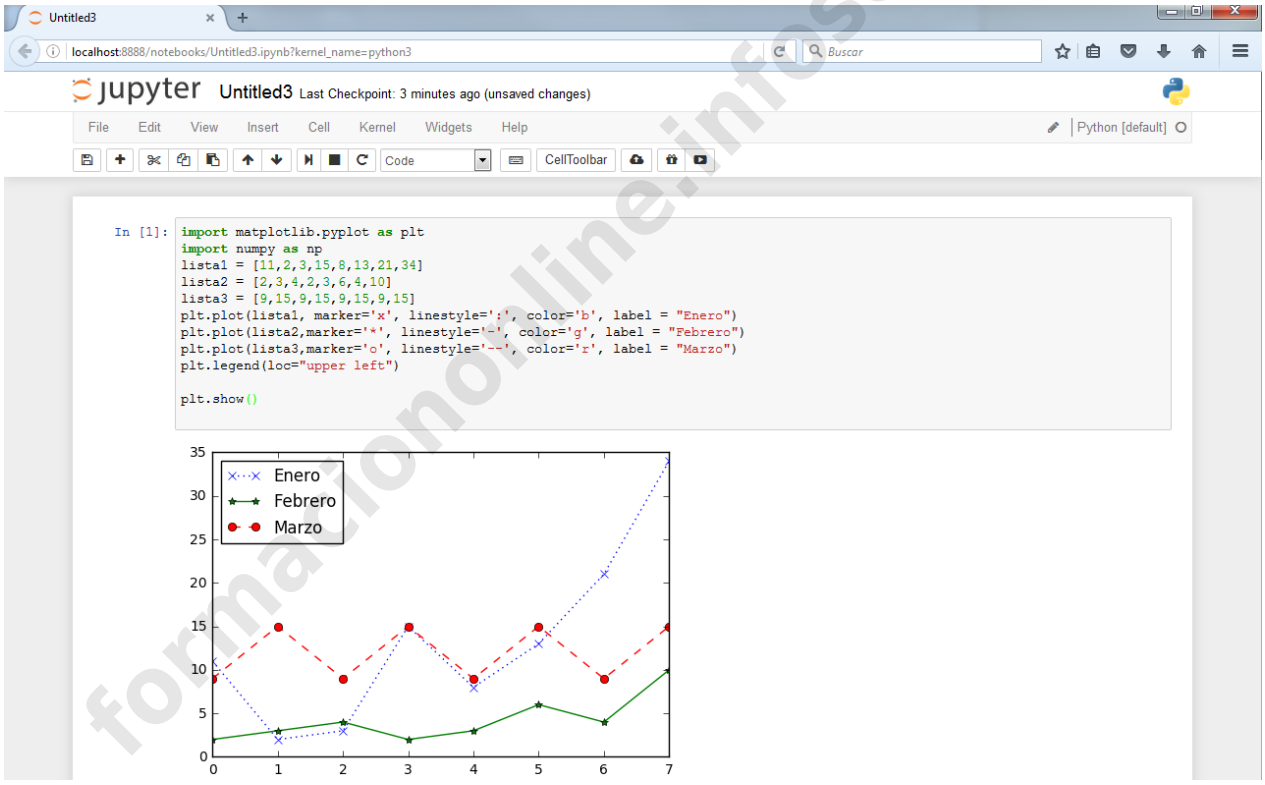
Podemos establecer diferentes estilos, marcadores y colores para las líneas de un gráfico mediante los siguientes argumentos de la **función plot**:

| Argumento | Valores que puede tomar |
|---|---|
| linestyle | • -, Línea solida<br>• --, Línea discontinua<br>• :, Línea punteada<br>• -., Línea punteada discontinua<br>• None, ninguna línea |

formaciononline.infosernt.com

| | |
|---|---|
| marker | • +, cruz<br>• ., punto<br>• o, círculo<br>• *, estrella<br>• p, pentágonos<br>• s, cuadrados<br>• x, tachados<br>• D, diamantes<br>• h, hexáganos<br>• ^, triángulo |
| color | • b, azul<br>• g, verde<br>• r, rojo<br>• c, cyan<br>• m, magenta<br>• y, amarillo<br>• k, negro<br>• w, blanco |

Mostramos un ejemplo a continuación:




Por último, mostramos a continuación otra serie de **métodos de utilidad** para trabajar con gráficos:

- plt.cla() : Borra toda la información relacionada con los ejes de un gráfico.
- plt.clf() : Borra por completo un gráfico.
- plt.close() : Termina el gráfico cerrando su ventana.

formaciononline.infosernt.com

- savefig(“archivo.png”) : Permite guardar el gráfico en formato .png.
- savefig(“archivo.pdf”) : Permite guardar el gráfico en formato .pdf.

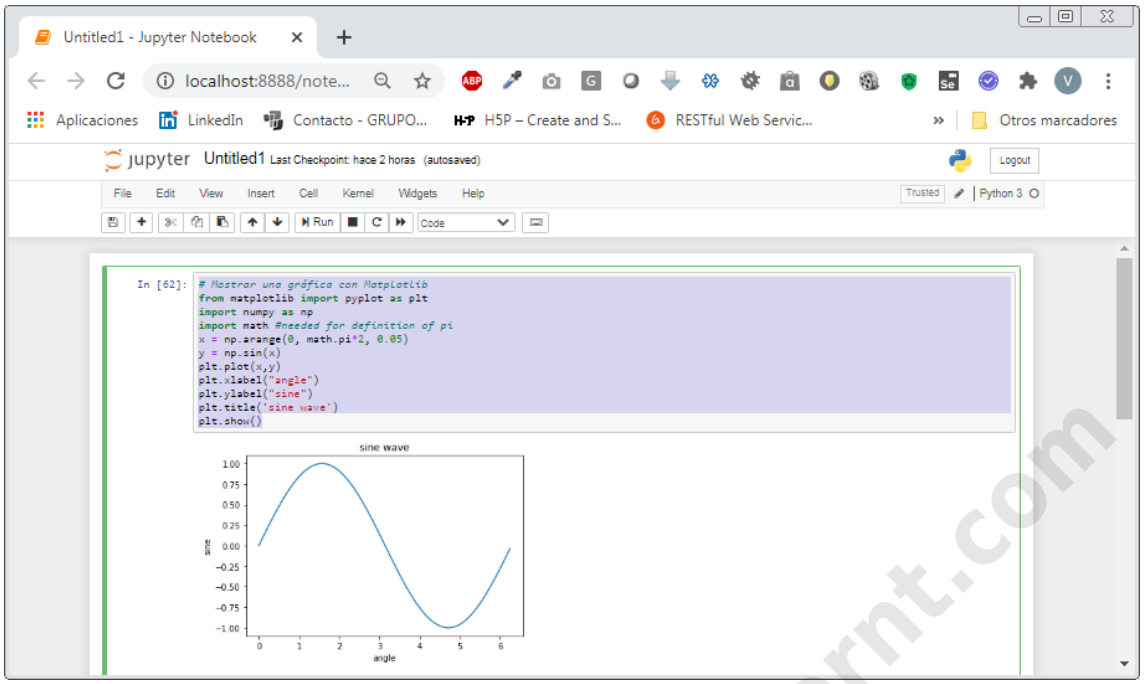
Para guardar los gráficos también podemos usar la pestaña de guardar de la ventana que se nos abre cuando trabajamos con IPython.

Como hemos visto, el paquete matplotlib y en concreto la librería pyplot, tiene multitud de opciones para dibujar gráficos que nos permiten visualizar el resultado del análisis de datos. En este punto hemos hecho una pequeña introducción a este paquete del que se puede obtener más documentación y ejemplos de uso de su sitio oficial.

Ejemplos de código con Matplotlib

```
# Mostrar una gráfica con Matplotlib

from matplotlib import pyplot as plt

import numpy as np

import math #needed for definition of pi

x = np.arange(0, math.pi*2, 0.05)

y = np.sin(x)

plt.plot(x,y)

plt.xlabel("angle")

plt.ylabel("sine")

plt.title('sine wave')

plt.show()
```

formaciononline.infosernt.com




# Gráfica con formato

from pylab import *

x = linspace(-3, 3, 30)

y = x**2

plot(x, y, 'r.')

show()

# Gráficas con varios formatos

from pylab import *

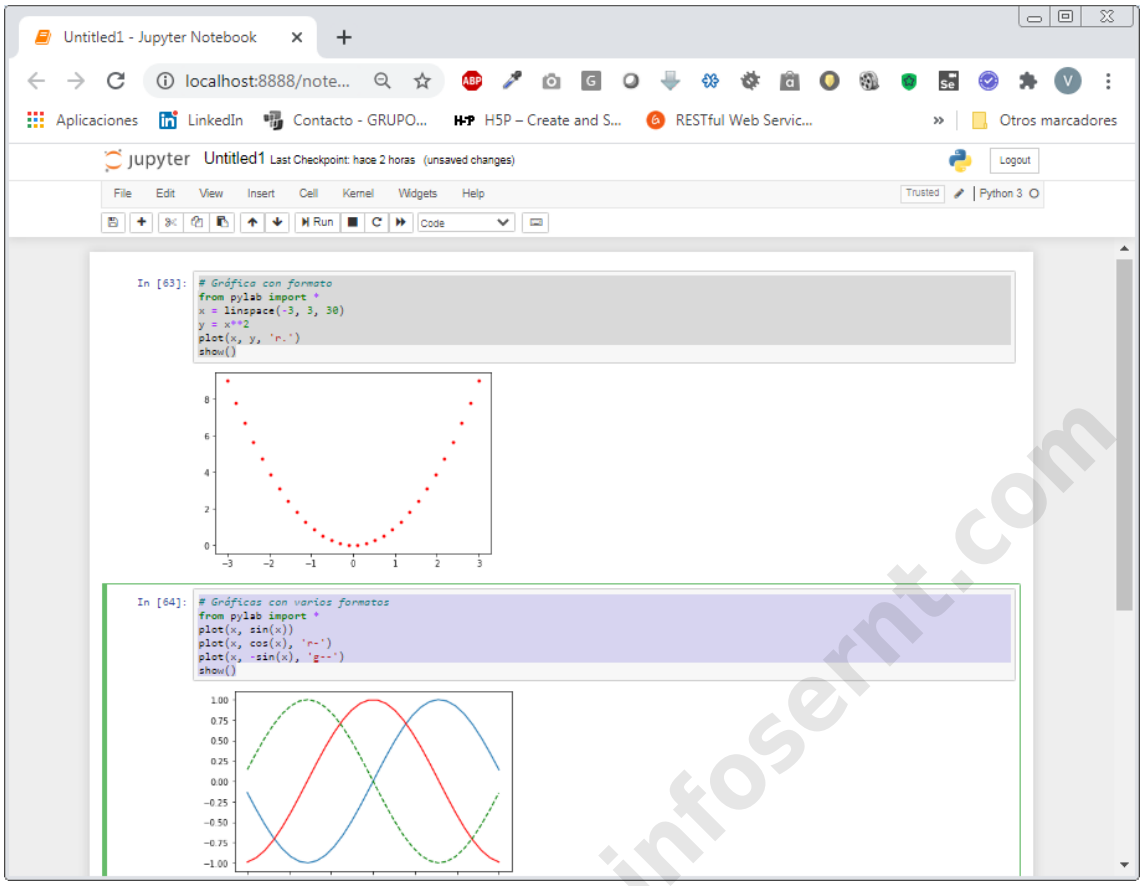
plot(x, sin(x))

plot(x, cos(x), 'r-')

plot(x, -sin(x), 'g--')

show()

formaciononline.infosernt.com




# Incluir varios subplots

import matplotlib.pyplot as plt

fig,a = plt.subplots(2,2)

import numpy as np

x = np.arange(1,5)

a[0][0].plot(x,x*x)

a[0][0].set_title('cuadrado')

a[0][1].plot(x,np.sqrt(x))

a[0][1].set_title('raíz cuadrada')

a[1][0].plot(x,np.exp(x))

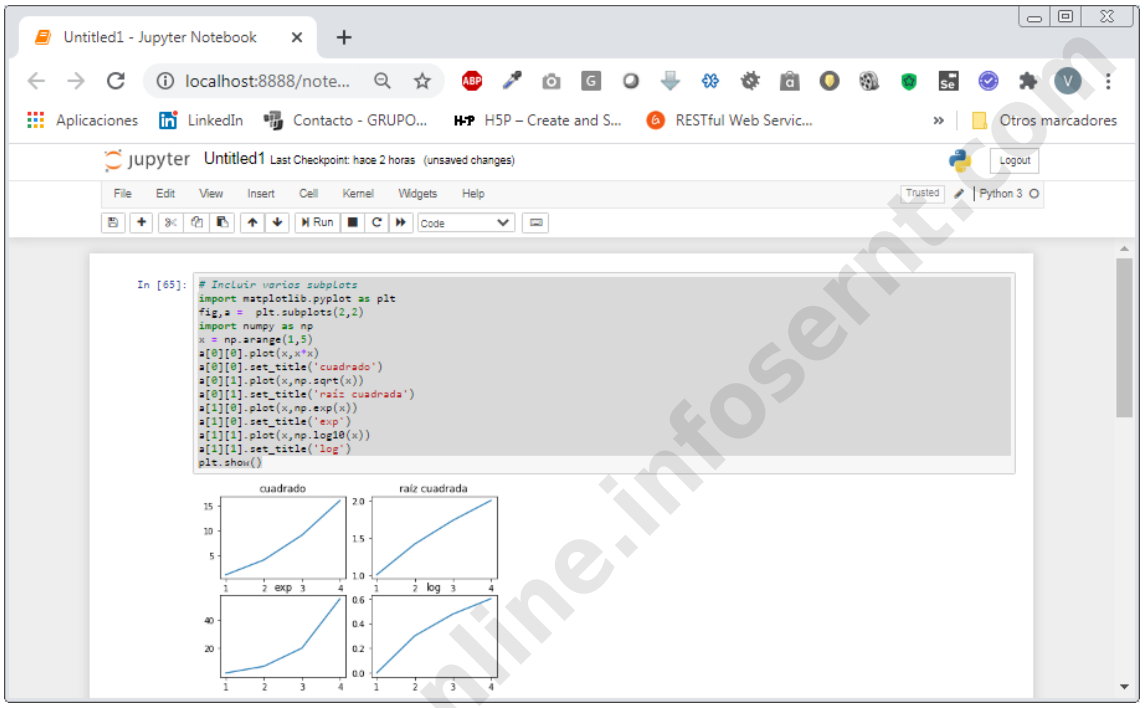
formaciononline.infosernt.com

a[1][0].set_title('exp')

a[1][1].plot(x,np.log10(x))

a[1][1].set_title('log')

plt.show()




# Modificar cuadrícula

import matplotlib.pyplot as plt

import numpy as np

fig, axes = plt.subplots(1,3, figsize = (12,4))

x = np.arange(1,11)

axes[0].plot(x, x**3, 'g',lw=2)

axes[0].grid(True)

axes[0].set_title('grid por defecto')

formaciononline.infosernt.com

axes[1].plot(x, np.exp(x), 'r')

axes[1].grid(color='b', ls = '-.', lw = 0.25)

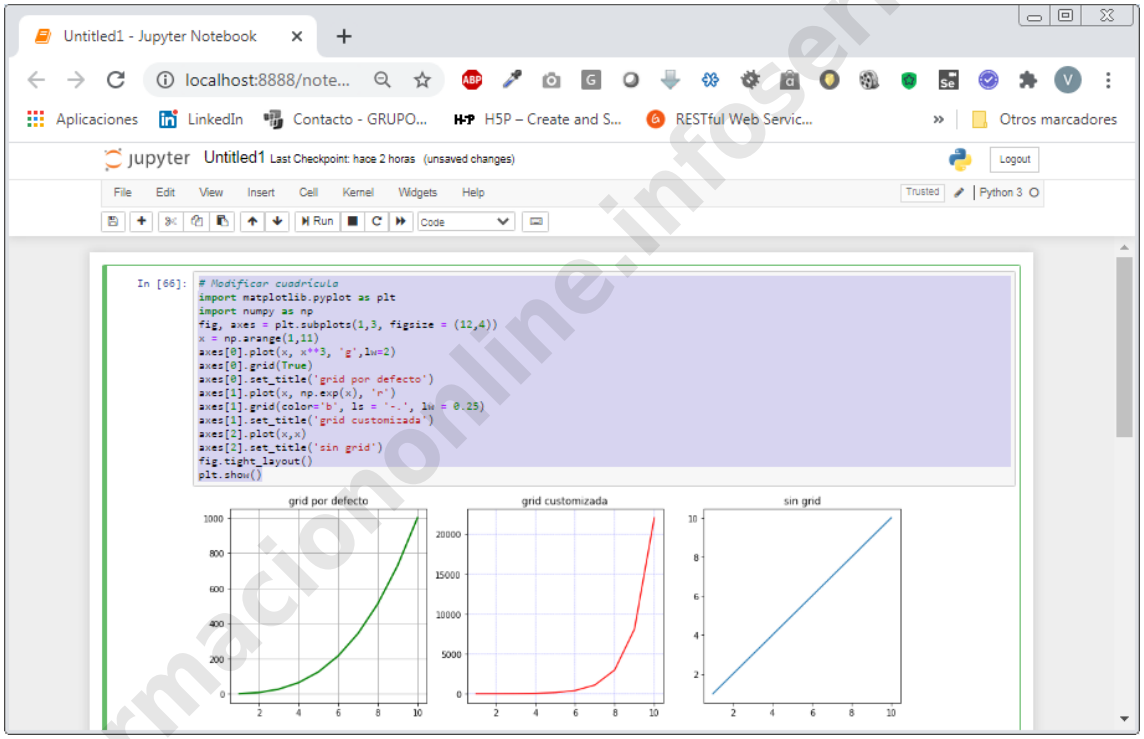
axes[1].set_title('grid customizada')

axes[2].plot(x,x)

axes[2].set_title('sin grid')

fig.tight_layout()

plt.show()




# Formatear ejes

import matplotlib.pyplot as plt

import numpy as np

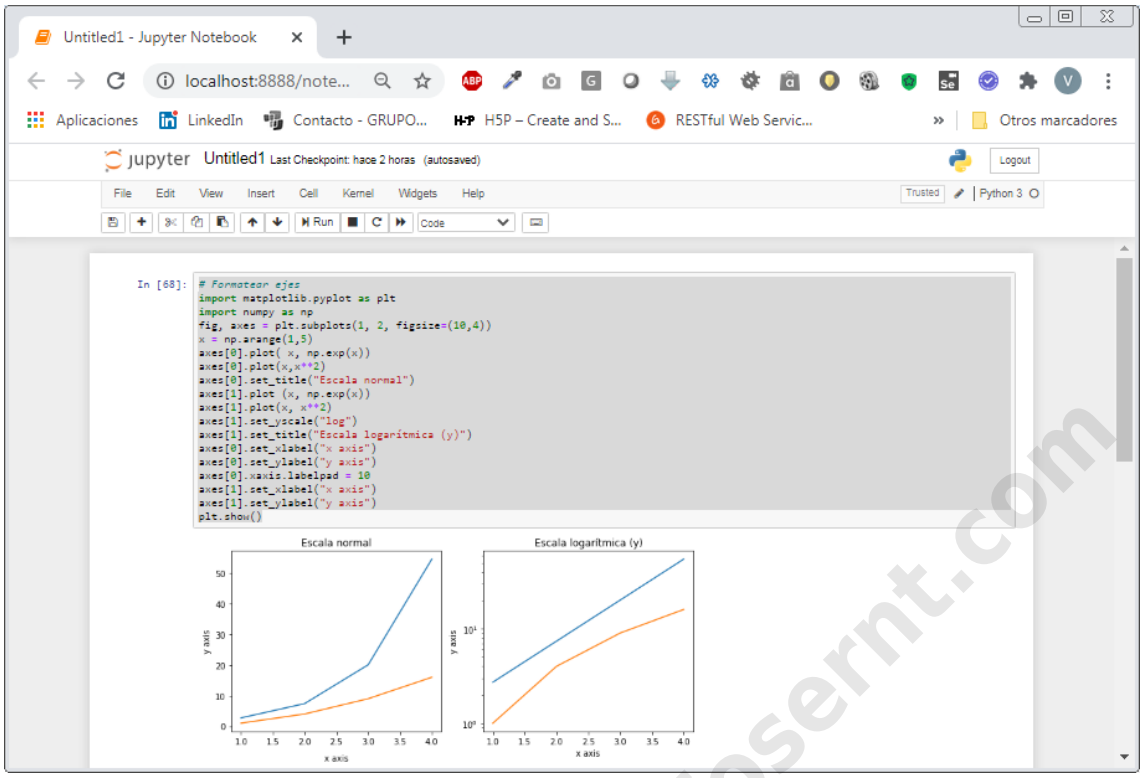
fig, axes = plt.subplots(1, 2, figsize=(10,4))

x = np.arange(1,5)

formaciononline.infosernt.com

```
axes[0].plot( x, np.exp(x))

axes[0].plot(x,x**2)

axes[0].set_title("Escala normal")

axes[1].plot (x, np.exp(x))

axes[1].plot(x, x**2)

axes[1].set_yscale("log")

axes[1].set_title("Escala logarítmica (y)")

axes[0].set_xlabel("x axis")

axes[0].set_ylabel("y axis")

axes[0].xaxis.labelpad = 10

axes[1].set_xlabel("x axis")

axes[1].set_ylabel("y axis")

plt.show()
```

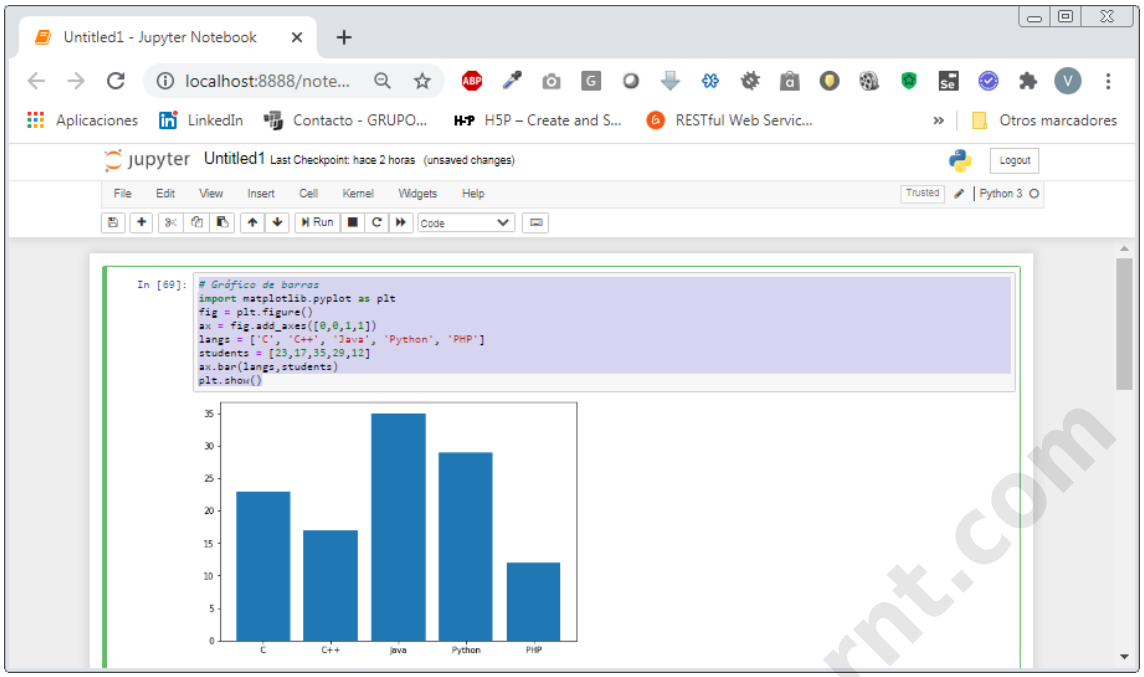
formaciononline.infosernt.com




# Gráfico de barras

```
import matplotlib.pyplot as plt

fig = plt.figure()

ax = fig.add_axes([0,0,1,1])

langs = ['C', 'C++', 'Java', 'Python', 'PHP']

students = [23,17,35,29,12]

ax.bar(langs,students)

plt.show()
```

formaciononline.infosernt.com




# Histograma

from matplotlib import pyplot as plt

import numpy as np

fig,ax = plt.subplots(1,1)

a = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])

ax.hist(a, bins = [0,25,50,75,100])

ax.set_title("histogram of result")

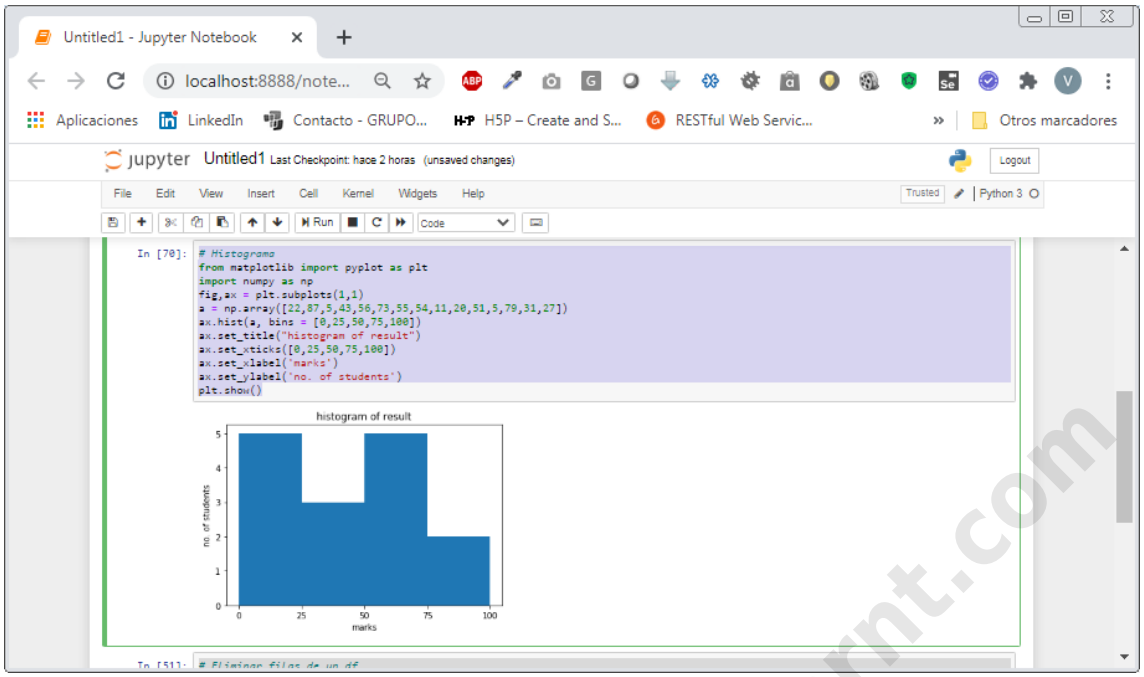
ax.set_xticks([0,25,50,75,100])

ax.set_xlabel('marks')

ax.set_ylabel('no. of students')

plt.show()

formaciononline.infosernt.com




# Gráfico de pie

from matplotlib import pyplot as plt

import numpy as np

fig = plt.figure()

ax = fig.add_axes([0,0,1,1])

ax.axis('equal')

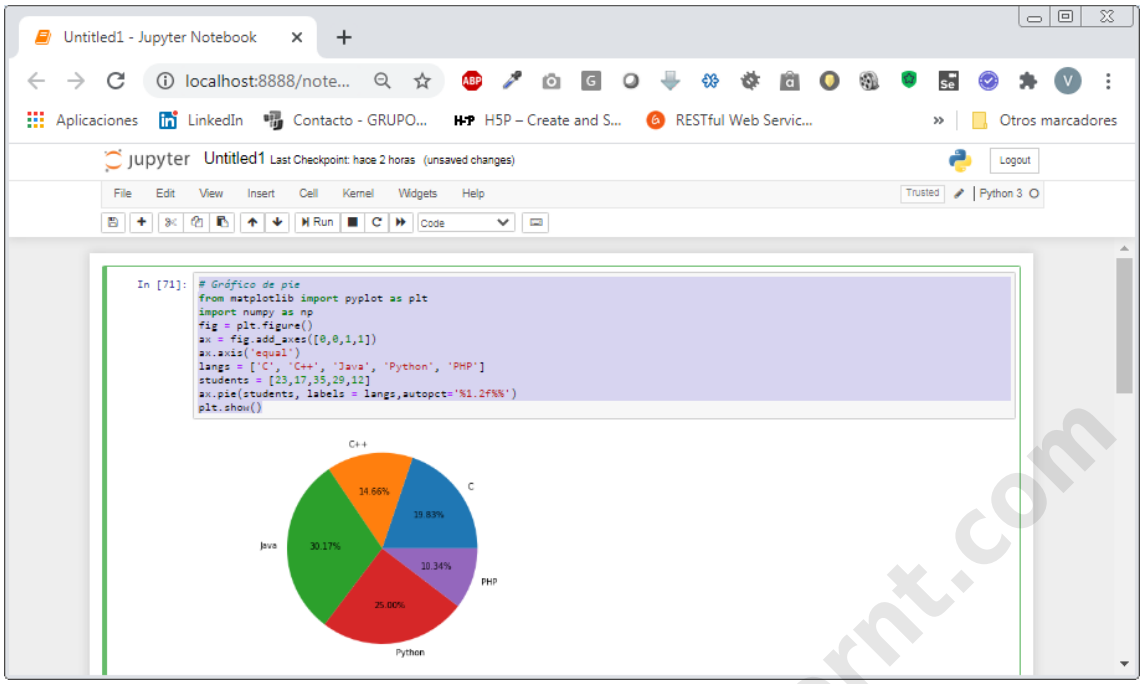
langs = ['C', 'C++', 'Java', 'Python', 'PHP']

students = [23,17,35,29,12]

ax.pie(students, labels = langs,autopct='%1.2f%%')

plt.show()

formaciononline.infosernt.com




# Surface plot

from mpl_toolkits import mplot3d

import numpy as np

import matplotlib.pyplot as plt

x = np.outer(np.linspace(-2, 2, 30), np.ones(30))

y = x.copy().T # transpose

z = np.cos(x ** 2 + y ** 2)

fig = plt.figure()

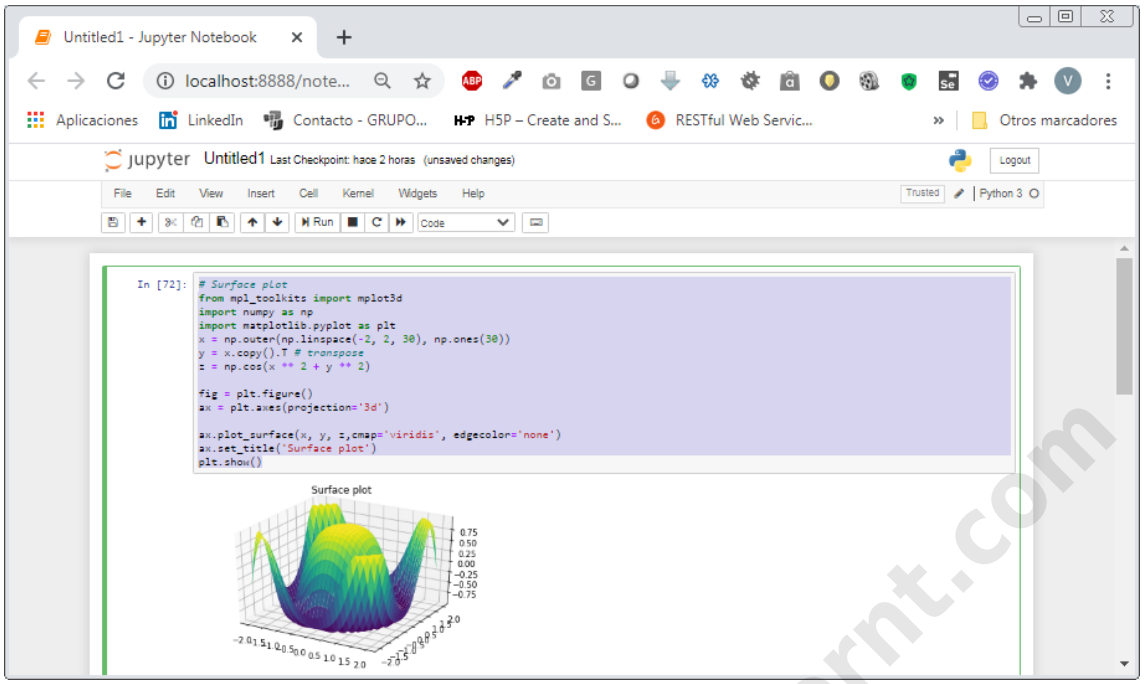
ax = plt.axes(projection='3d')

ax.plot_surface(x, y, z,cmap='viridis', edgecolor='none')

ax.set_title('Surface plot')

plt.show()

formaciononline.infosernt.com




**Como cualquier otra librería de Python, cuenta con un sitio oficial, a través del cual se generan actualizaciones y documentación, así como tutoriales relacionados con el uso de las funciones que implementa este paquete.**

| |
|---|
| Verdadero. |
| Falso. |

**Es una biblioteca de Python:**

| |
|---|
| Theano. |
| Google TensorFlow. |

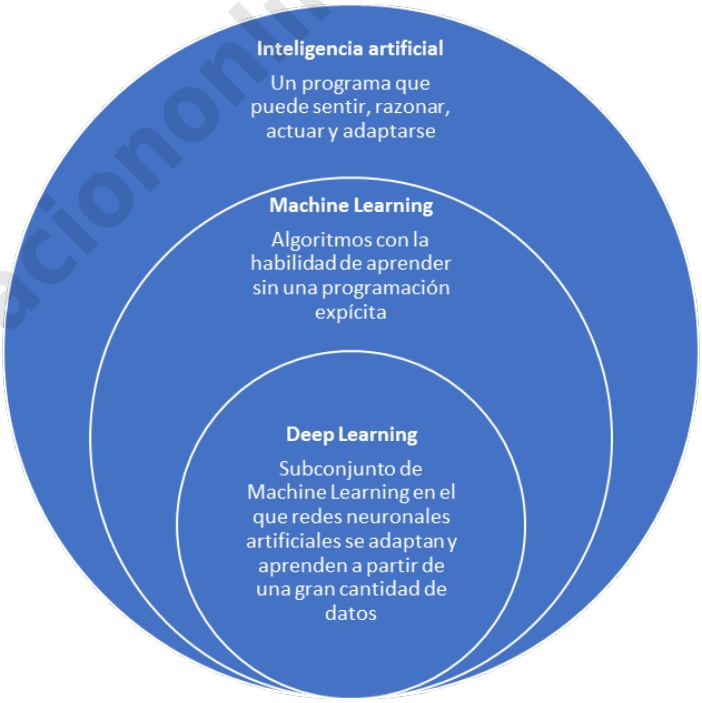
formaciononline.infosernt.com

# Machine Learning. Introducción.

La **inteligencia artificial** es la tecnología que permite a las computadoras hacer cosas que alguna vez fueron solo el dominio de los humanos. Por ejemplo, las computadoras siempre han podido calcular. Con IA, pueden aprender y sacar conclusiones.

El aprendizaje automático es una **aplicación de inteligencia artificial** (IA) que proporciona a los sistemas la capacidad de aprender y mejorar automáticamente a partir de la experiencia sin ser programado explícitamente. El aprendizaje automático se centra en el desarrollo de programas informáticos que pueden acceder a los datos y utilizarlos para aprender por sí mismos.

La IA puede dividirse aproximadamente en dos disciplinas. Estos son el aprendizaje automático y el aprendizaje profundo. El **aprendizaje automático (machine learning)** implica la creación de computadoras y software que pueden aprender de los datos y luego aplicar ese conocimiento a nuevos conjuntos de datos. El **aprendizaje profundo (deep learning)** crea redes neuronales, diseñadas para parecerse al cerebro humano. El aprendizaje profundo se utiliza para procesar datos, como sonidos e imágenes.




El **proceso de aprendizaje** comienza con observaciones o datos, como ejemplos, experiencia directa o instrucción, para buscar patrones en los datos y tomar mejores decisiones en el futuro en

formaciononline.infosernt.com

función de los ejemplos que proporcionamos. El objetivo principal es permitir que las computadoras aprendan automáticamente sin intervención o asistencia humana y ajustar las acciones en consecuencia.

Es una categoría de algoritmo que permite que las aplicaciones de software sean más precisas para predecir resultados sin ser programadas explícitamente. La premisa básica del aprendizaje automático es crear algoritmos que puedan recibir datos de entrada y utilizar análisis estadísticos para predecir un resultado mientras se actualizan los resultados a medida que se dispone de nuevos datos.

Debido a las nuevas tecnologías informáticas, el aprendizaje automático de hoy no es como el aprendizaje automático del pasado. Nació del reconocimiento de patrones y la teoría de que las computadoras pueden aprender sin ser programadas para realizar tareas específicas.

Los investigadores interesados en la **inteligencia artificial** querían ver si las computadoras podían aprender de los datos. El aspecto iterativo del aprendizaje automático es importante porque a medida que los modelos están expuestos a nuevos datos, pueden adaptarse de forma independiente. Aprenden de cálculos anteriores para producir decisiones y resultados confiables y repetibles. Es una ciencia que no es nueva, pero que ha ganado un nuevo impulso.

El **aprendizaje automático (ML)** es una categoría de algoritmo que permite que las aplicaciones de software sean más precisas para predecir resultados sin ser programadas explícitamente. La premisa básica del aprendizaje automático es crear algoritmos que puedan recibir datos de entrada y utilizar análisis estadísticos para predecir un resultado mientras se actualizan los resultados a medida que se dispone de nuevos datos.

Los siguientes pasos definen el proceso de Machine Learning:

- Identifica conjuntos de datos relevantes y los prepara para su análisis.
- Elige el tipo de algoritmo a usar.
- Crea un modelo analítico basado en el algoritmo utilizado.
- Entrena el modelo en conjuntos de datos de prueba y lo revisa según sea necesario.
- Ejecuta el modelo para generar puntuaciones de prueba.

formaciononline.infosernt.com

Los procesos involucrados en el aprendizaje automático son similares a los de la minería de datos y el modelado predictivo. Ambos requieren buscar en los datos para buscar patrones y ajustar las acciones del programa en consecuencia. Muchas personas están familiarizadas con el aprendizaje automático al comprar en Internet y recibir anuncios relacionados con su compra. Esto sucede porque los motores de recomendación utilizan el aprendizaje automático para personalizar la entrega de anuncios en línea en casi tiempo real.

Más allá del marketing personalizado, otros casos de uso de aprendizaje automático comunes incluyen detección de fraude, filtrado de spam, detección de amenazas a la seguridad de la red, mantenimiento predictivo y creación de noticias.

## Clasificación de algoritmos de aprendizaje automático

Los algoritmos de aprendizaje automático a menudo se clasifican como supervisados o no supervisados.

Los **algoritmos de aprendizaje automático supervisados** pueden aplicar lo aprendido en el pasado a nuevos datos utilizando ejemplos etiquetados para predecir eventos futuros. Requieren un científico de datos o un analista de datos con habilidades de aprendizaje automático para proporcionar tanto la entrada como la salida deseada, además de proporcionar comentarios sobre la precisión de las predicciones durante el entrenamiento del algoritmo.

Los científicos de datos determinan qué variables o características, el modelo debe analizar y usar para desarrollar predicciones. Una vez que se completa la capacitación, el algoritmo aplicará lo aprendido a los nuevos datos.

A medida que los **datos de entrada** se introducen en el modelo, ajusta sus pesos hasta que el modelo se haya ajustado adecuadamente. Esto ocurre como parte del proceso de validación cruzada para garantizar que el modelo evite el sobreajuste (overfitting) o el subajuste (underfitting).

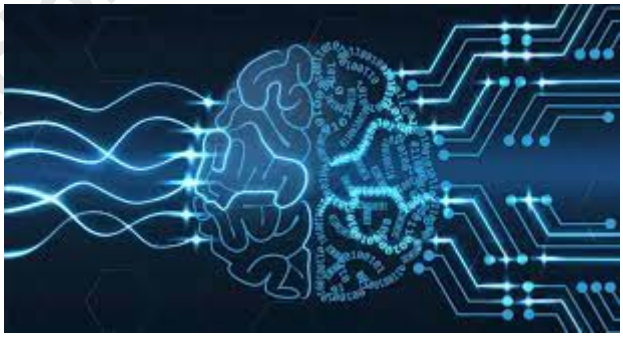
A partir del análisis de un conjunto de datos de entrenamiento conocido, el algoritmo de aprendizaje produce una función inferida para hacer predicciones sobre los valores de salida. El sistema puede proporcionar objetivos para cualquier entrada nueva después de una capacitación suficiente. El algoritmo de aprendizaje también puede comparar su salida con la salida correcta e intencionada y encontrar errores para modificar el modelo en consecuencia.

formaciononline.infosernt.com

Algunos métodos utilizados en el aprendizaje supervisado incluyen redes neuronales, bayes naïve, regresión lineal, regresión logística, random forest, máquina de vectores de soporte (SVM) y más.

Por el contrario, los **algoritmos de aprendizaje automático no supervisados** se utilizan cuando la información utilizada para entrenar no está clasificada ni etiquetada. El aprendizaje no supervisado estudia cómo los sistemas pueden inferir una función para describir una estructura oculta a partir de datos no etiquetados. El sistema no encuentra la salida correcta, pero explora los datos y puede extraer inferencias de conjuntos de datos para describir estructuras ocultas a partir de datos no etiquetados.

Estos algoritmos no necesitan ser entrenados con los datos de resultados deseados. En cambio, utilizan un enfoque iterativo llamado **aprendizaje profundo** (Deep learning) para revisar los datos y llegar a conclusiones. Los algoritmos de aprendizaje no supervisados, también llamados **redes neuronales**, se utilizan para tareas de procesamiento más complejas que los sistemas de aprendizaje supervisados, incluidos el reconocimiento de imágenes, la conversión de voz a texto y lenguaje natural. Estas redes neuronales funcionan combinando millones de ejemplos de datos de entrenamiento e identificando automáticamente correlaciones a menudo sutiles entre muchas variables. Una vez entrenado, el algoritmo puede usar su banco de asociaciones para interpretar nuevos datos. Estos algoritmos solo se han vuelto viables en la era de los grandes datos, ya que requieren grandes cantidades de datos de entrenamiento.



La capacidad del aprendizaje automático no supervisado para descubrir similitudes y diferencias en la información lo convierten en la solución ideal para el análisis exploratorio de datos, estrategias de venta cruzada, segmentación de clientes, reconocimiento de imágenes y patrones. También se utiliza para reducir el número de características en un modelo a través del proceso de reducción de la dimensionalidad. El análisis de componentes principales (PCA) y la descomposición de valores

formaciononline.infosernt.com

singulares (SVD) son dos enfoques habituales para lograr esto. Otros algoritmos utilizados en el aprendizaje no supervisado incluyen redes neuronales, k-means clustering, métodos de clustering probabilístico, etc.

Los **algoritmos de aprendizaje automático semi supervisados** se encuentran en algún punto intermedio entre el aprendizaje supervisado y el no supervisado, ya que utilizan datos etiquetados y no etiquetados para el entrenamiento, generalmente una pequeña cantidad de datos etiquetados y una gran cantidad de datos no etiquetados. Los sistemas que utilizan este método pueden mejorar considerablemente la precisión del aprendizaje. Por lo general, el aprendizaje semi-supervisado se elige cuando los datos etiquetados adquiridos requieren recursos calificados y relevantes para capacitarlos / aprender de ellos. De lo contrario, la adquisición de datos no etiquetados generalmente no requiere recursos adicionales.

Los **algoritmos de aprendizaje automático de refuerzo** son un método de aprendizaje que interactúa con su entorno mediante la producción de acciones y descubre errores o recompensas. La búsqueda de prueba y error y la recompensa retrasada son las características más relevantes del aprendizaje por refuerzo. Este método permite que las máquinas y los agentes de software determinen automáticamente el comportamiento ideal dentro de un contexto específico para maximizar su rendimiento. Se requiere una retroalimentación de recompensa simple para que el agente aprenda qué acción es la mejor; Esto se conoce como la señal de refuerzo.

El algoritmo de aprendizaje de una red neuronal puede ser supervisado o no supervisado. Se dice que una red neuronal aprende de forma supervisada si ya se conoce la salida deseada. Durante el aprendizaje, uno de los patrones de entrada se da a la capa de entrada de la red. Las redes neuronales que aprenden sin supervisión no tienen tales resultados objetivo.

El aprendizaje no supervisado es el **Santo Grial del Deep Learning**. El objetivo del aprendizaje no supervisado es crear sistemas generales que puedan ser entrenados con pocos datos. Hoy en día, los modelos de aprendizaje profundo se entrenan a partir de grandes conjuntos de datos supervisados. Esto implica que para cada dato, hay una etiqueta correspondiente.

El sistema de aprendizaje de un algoritmo de aprendizaje automático se divide en tres partes principales:

formaciononline.infosernt.com

- **Un proceso de decisión:** en general, los algoritmos de aprendizaje automático se utilizan para hacer una predicción o clasificación. Basado en algunos datos de entrada, que pueden ser etiquetados o no etiquetados, nuestro algoritmo producirá una estimación sobre un patrón en los datos.
- **Una función de error:** una función de error sirve para evaluar la predicción del modelo. Si hay ejemplos conocidos, una función de error puede hacer una comparación para evaluar la precisión del modelo.
- **Un proceso de optimización del modelo:** si el modelo puede ajustarse mejor a los puntos de datos en el conjunto de entrenamiento, entonces los pesos se ajustan para reducir la discrepancia entre el ejemplo conocido y la estimación del modelo. El algoritmo repetirá este proceso de evaluación y optimización, actualizando los pesos de forma autónoma hasta que se haya alcanzado un umbral de precisión.

El aprendizaje automático permite el análisis de cantidades masivas de datos. Si bien generalmente ofrece resultados más rápidos y precisos para identificar oportunidades rentables o riesgos peligrosos, también puede requerir tiempo y recursos adicionales para entrenarlo adecuadamente. La combinación de aprendizaje automático con IA y tecnologías cognitivas puede hacerlo aún más efectivo en el procesamiento de grandes volúmenes de información.

## Ejemplos de aprendizaje automático

El aprendizaje automático se está utilizando en una amplia gama de aplicaciones hoy en día. Uno de los ejemplos más conocidos es el **News Feed de Facebook**. News Feed utiliza el aprendizaje automático para personalizar el feed de cada miembro. Si un miembro con frecuencia deja de desplazarse para leer o darle me gusta a las publicaciones de un amigo en particular, el servicio de noticias comenzará a mostrar más de la actividad de ese amigo antes en el feed.

formaciononline.infosernt.com

El software está usando análisis estadístico y análisis predictivo para identificar patrones en los datos del usuario y usar esos patrones para llenar el News Feed. Si el miembro ya no se detiene para leer, dar me gusta o comentar las publicaciones de un amigo, se incluirán nuevos datos en el conjunto de datos y las Noticias se ajustarán en consecuencia.

El aprendizaje automático también está llegando a una variedad de aplicaciones empresariales. Los sistemas de gestión de relaciones con los clientes (CRM) utilizan modelos de aprendizaje para analizar el correo electrónico y solicitar a los miembros del equipo de ventas que respondan primero a los mensajes más importantes. Los sistemas más avanzados pueden incluso recomendar respuestas potencialmente efectivas. Los proveedores de inteligencia empresarial (BI) y análisis utilizan el aprendizaje automático en su software para ayudar a los usuarios a identificar automáticamente los puntos de datos potencialmente importantes. Los sistemas de recursos humanos (RR. HH.) Utilizan modelos de aprendizaje para identificar las características de los empleados efectivos y se basan en este conocimiento para encontrar los mejores candidatos para puestos vacantes.

El aprendizaje automático también juega un papel importante en los **coches sin conductor**. Las redes neuronales de aprendizaje profundo se utilizan para identificar objetos y determinar acciones óptimas para conducir con seguridad un vehículo por la carretera.

formaciononline.infosernt.com

La tecnología de asistente virtual también funciona a través del aprendizaje automático. Los asistentes inteligentes combinan varios modelos de aprendizaje profundo para interpretar el lenguaje natural, obteniendo el contexto relevante, como el horario personal del usuario o las preferencias previamente definidas, y tomar una acción, como reservar un vuelo u obtener instrucciones de manejo.

Relaciona los elementos de la columna Derecha con la columna Izquierda

| | | | |
|---|---|---|---|
| *El aprendizaje automático* | 1 | | Es la tecnología que permite a las computadoras hacer cosas que alguna vez fueron solo el dominio de los humanos. Por ejemplo, las computadoras siempre han podido calcular. |
| *La inteligencia artificial* | 2 | | Proporciona a los sistemas la capacidad de aprender y mejorar automáticamente a partir de la experiencia sin ser programado explícitamente. |

formaciononline.infosernt.com

# Recuerda

[[[Elemento Multimedia]]]

formaciononline.infosernt.com

# Autoevaluación

**¿Qué devuelve el método keys() sobre un diccionario en Python?**

Devuelve la posición de la clave que se pasa por argumento.

Devuelve el dato correspondiente a la clave pasada por argumento.

Devuelve una lista con todas las claves, en un orden arbitrario, del diccionario.

**¿Qué significa que Python es un lenguaje interpretado?**

Abstrae el problema de manejo de memoria u otros detalles de bajo nivel.

No existen compilaciones separadas y pasos de ejecución. Solo es necesario ejecutar el código fuente.

Puede ser modificado por la comunidad en el caso de necesitar nuevas funciones o mejorar las existentes.

**Indica si la siguiente afirmación es verdadera o falsa:"Python es un lenguaje de programación interpretado multiparadigma, que soporta programación orientada a objetos, programación imperativa y en menor medida programación funcional"**

Verdadero.

Falso.

formaciononline.infosernt.com

**¿Qué nos devuelve la ejecución de la siguiente instrucción en Python conda list?**

Una lista con todos los paquetes y sus versiones que tenemos instalados.

Información referente a la versión de Anaconda que estamos utilizando.

Información referente a los paquetes que no tenemos aún instalados pero si disponibles para su descarga.

**El aprendizaje supervisado...**

Predice valores a partir de datos etiquetados.

Trabaja con datos no estructurados principalmente.

Necesita datos sin etiquetar.

formaciononline.infosernt.com